Backtracking

Backtracking

- Suponha que você tenha que tomar uma série de *decisões* a partir de um conjunto de *opções*, onde:



Backtracking



- Suponha que você tenha que tomar uma série de *decisões* a partir de um conjunto de *opções*, onde:
  - Não há informação suficiente para saber qual a melhor escolha.
  - Cada decisão leva a um novo conjunto de opções.
  - Cada sequência de decisões (possivelmente mais de uma) pode ser uma solução para o seu problema.

Backtracking



- Suponha que você tenha que tomar uma série de *decisões* a partir de um conjunto de *opções*, onde:
  - Não há informação suficiente para saber qual a melhor escolha.
  - Cada decisão leva a um novo conjunto de opções.
  - Cada sequência de decisões (possivelmente mais de uma) pode ser uma solução para o seu problema.

Backtracking



- Suponha que você tenha que tomar uma série de *decisões* a partir de um conjunto de *opções*, onde:
  - Não há informação suficiente para saber qual a melhor escolha.
  - Cada decisão leva a um novo conjunto de opções.
  - Cada sequência de decisões (possivelmente mais de uma) pode ser uma solução para o seu problema.

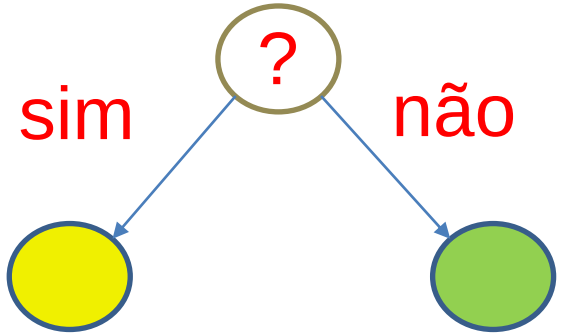
Backtracking

Podemos utilizar a técnica Backtracking



Backtracking

Maneira sistemática de testar várias sequências de decisões, até encontrar a que fornece uma solução ou a melhor solução ótima. É uma técnica enumerativa.



Backtracking

Usada quando precisamos encontrar soluções para problemas para os quais não se conhece um método eficiente.

Em geral, utiliza recursividade.



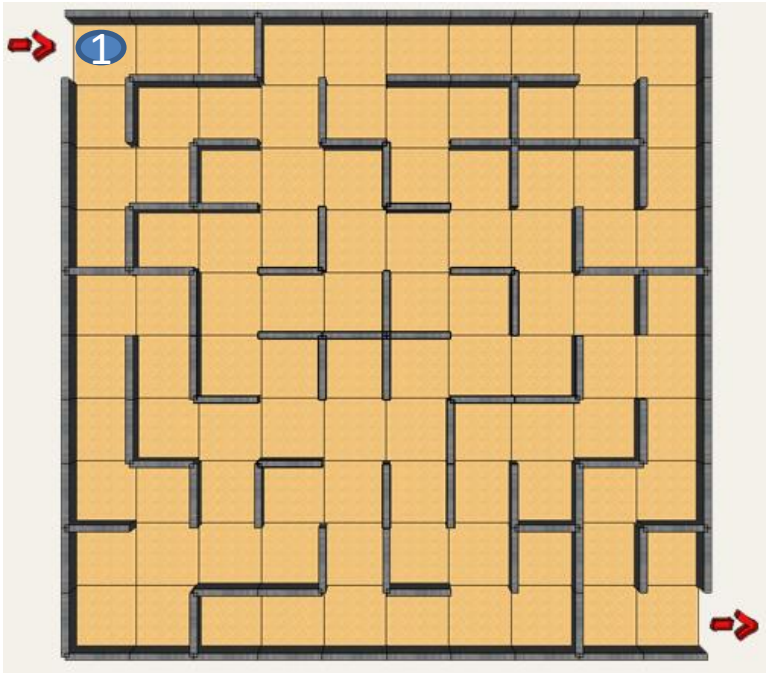
Backtracking

Faz um percurso em profundidade em um grafo implícito (árvore de decisão).

Exemplo - Labirinto

1

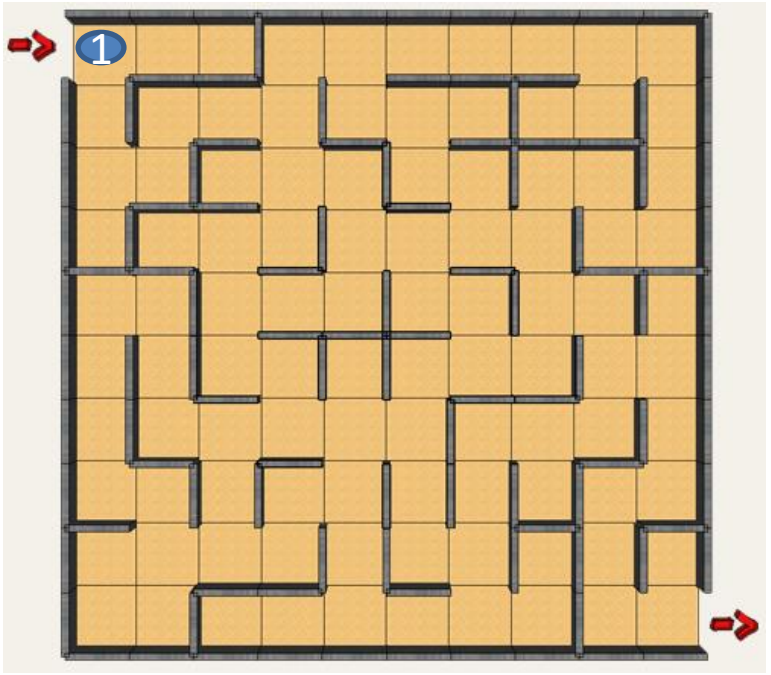
Duas opções:
- Ir em frente
- Ir para a direita



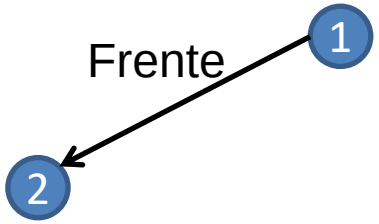
Exemplo - Labirinto

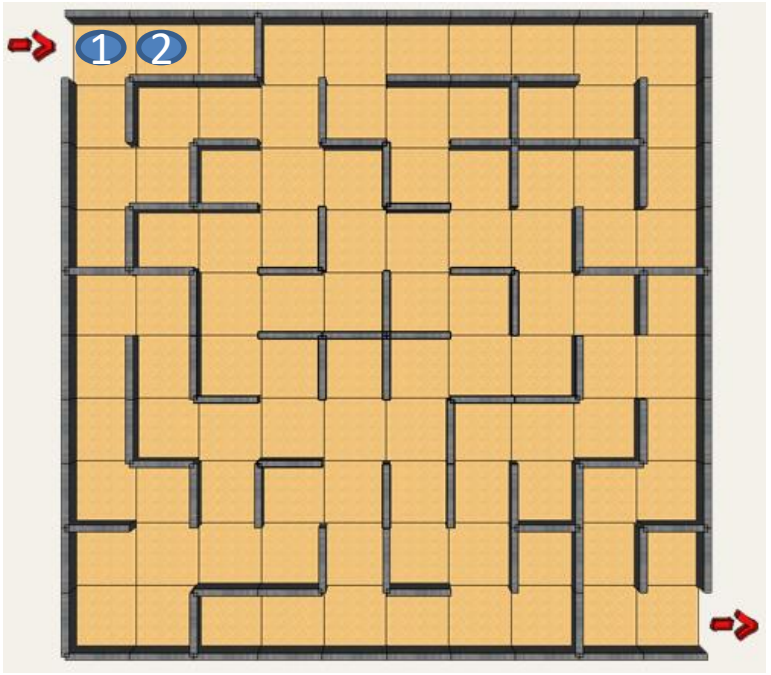
1

Duas opções:
- Ir em frente
- Ir para a direita

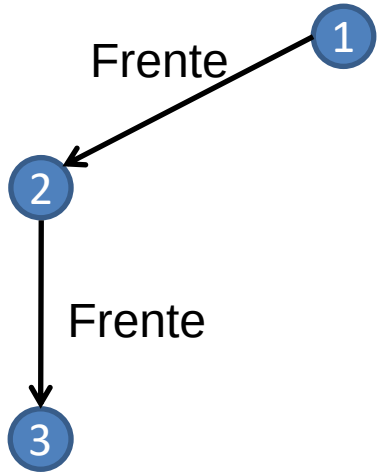


Backtracking - Labirinto

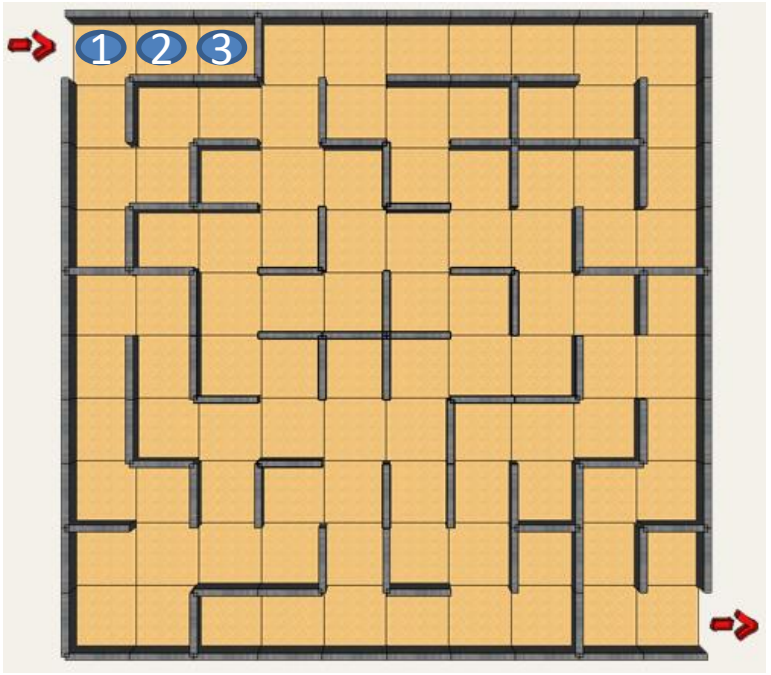




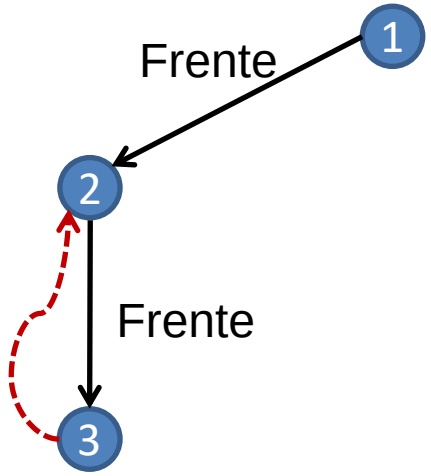
Backtracking - Labirinto

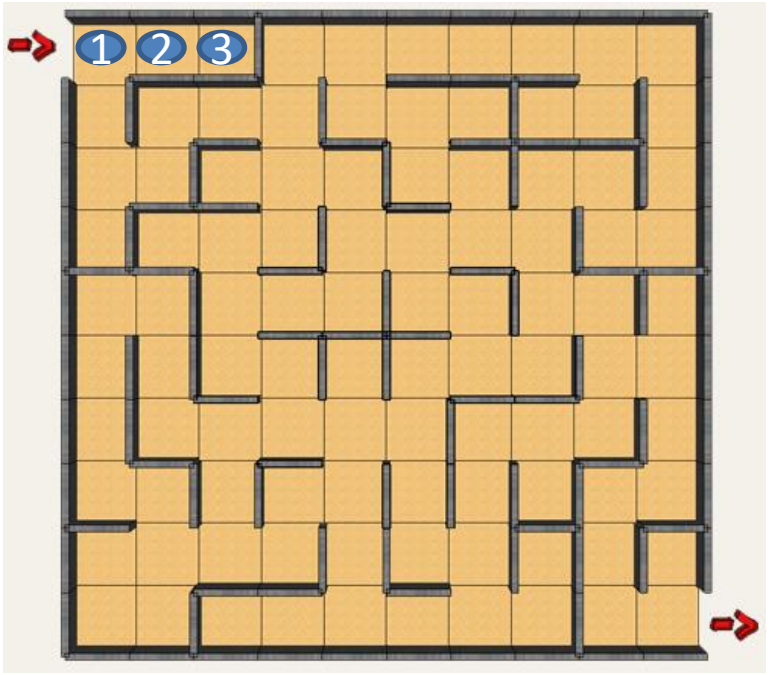


Não chegou ao destino, voltar atrás



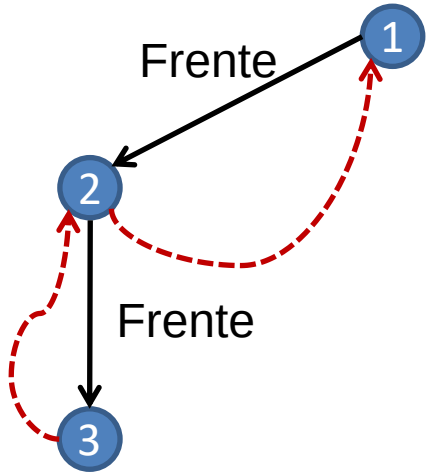
Backtracking - Labirinto

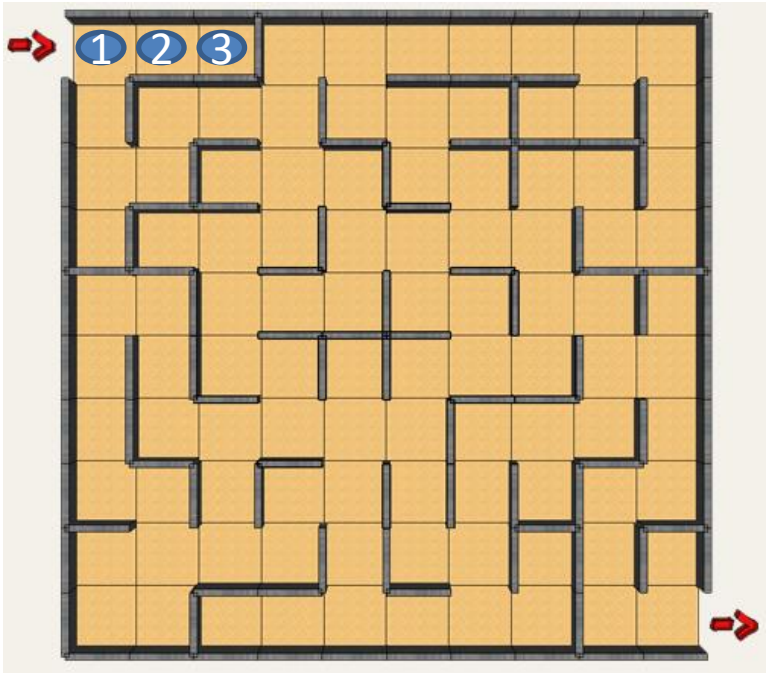




Backtracking

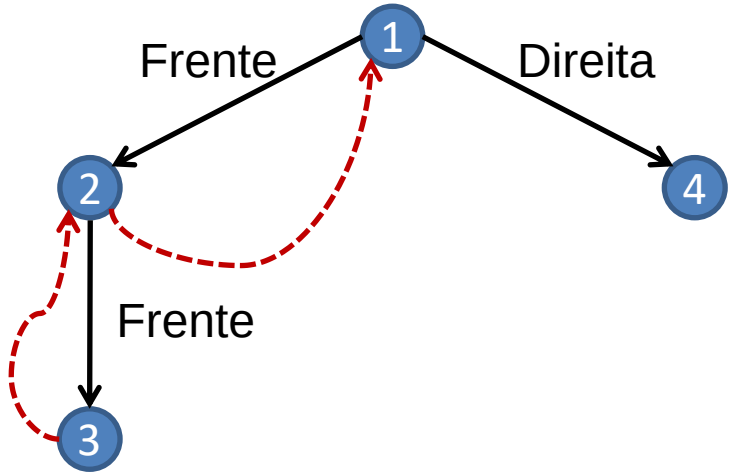
Backtracking - Labirinto

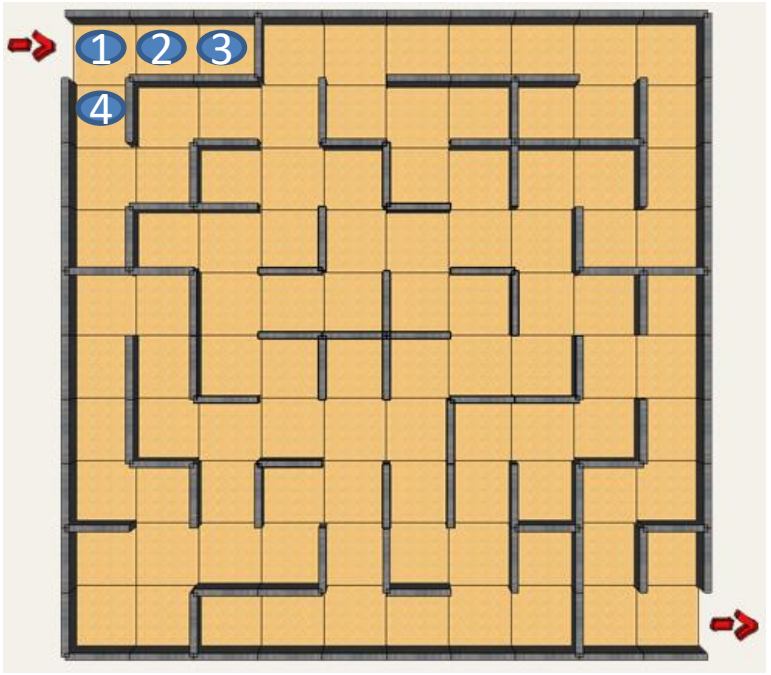




Backtracking

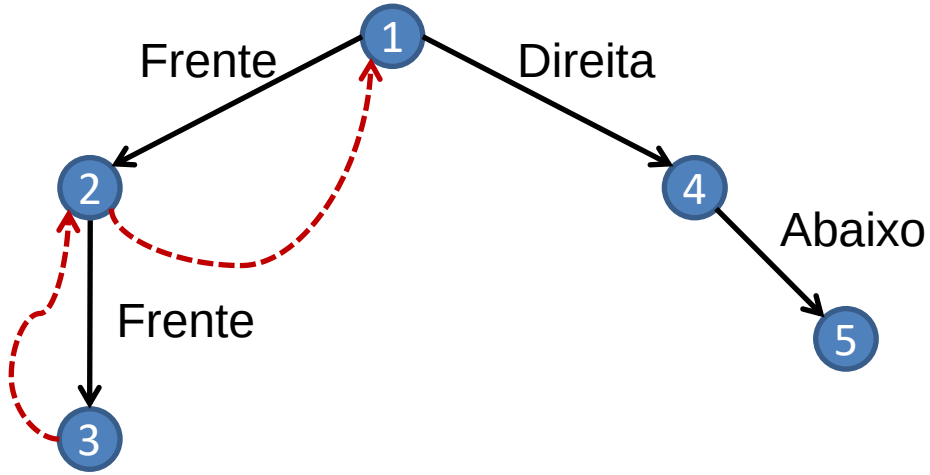
Backtracking - Labirinto

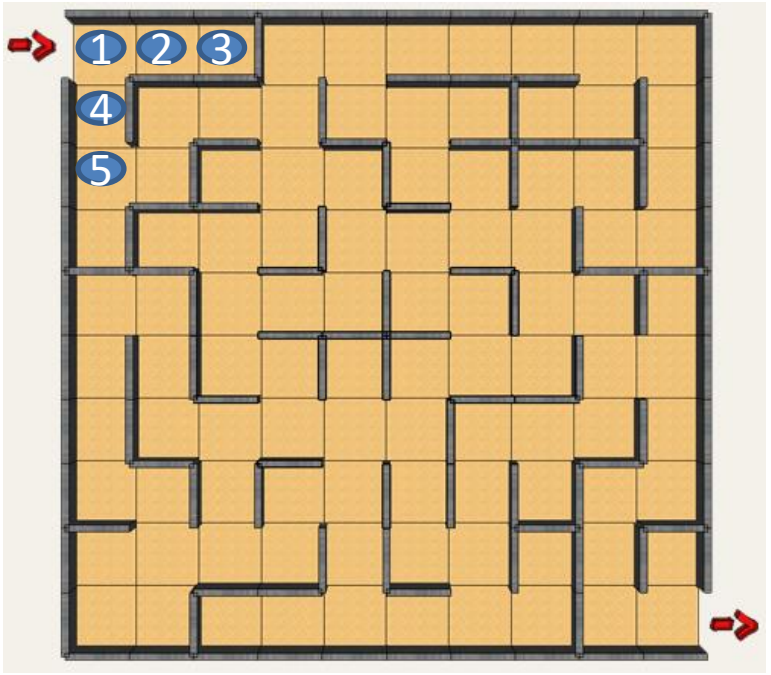




Backtracking

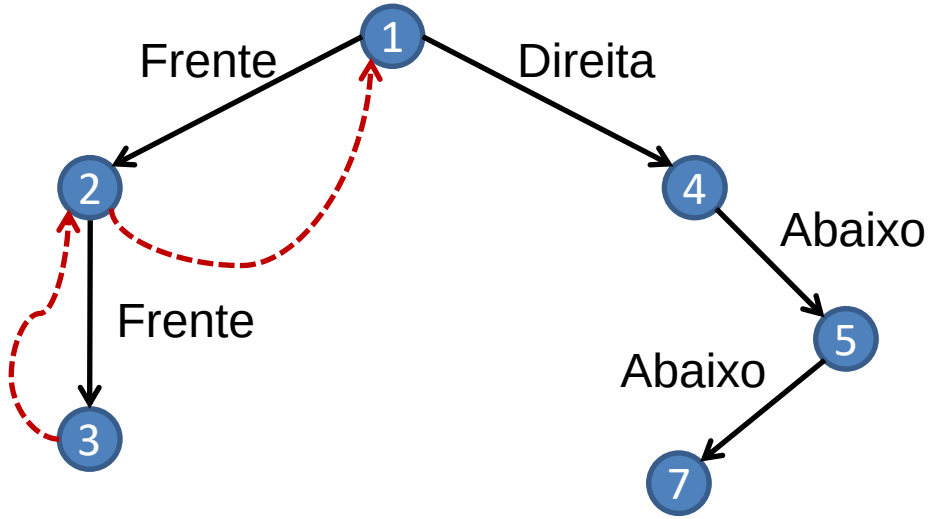
Backtracking - Labirinto

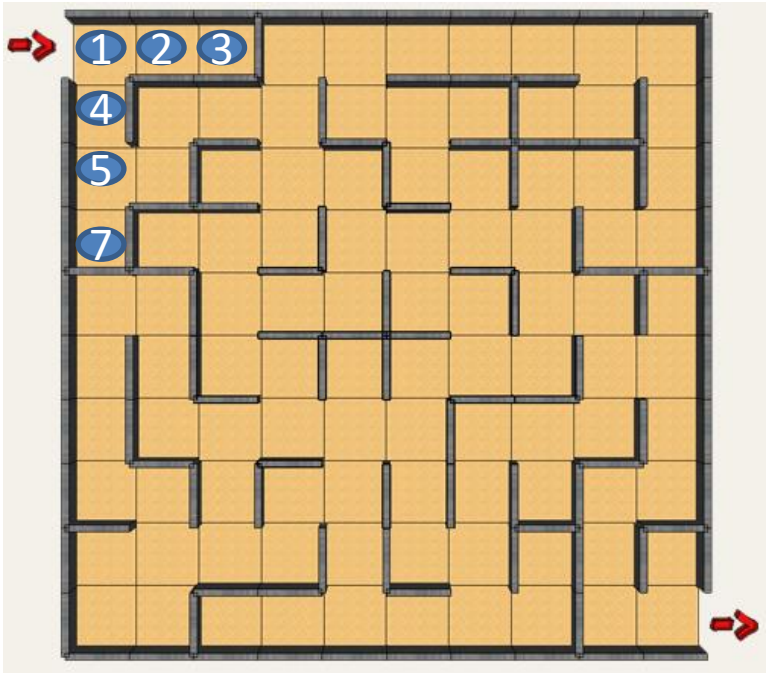




Backtracking

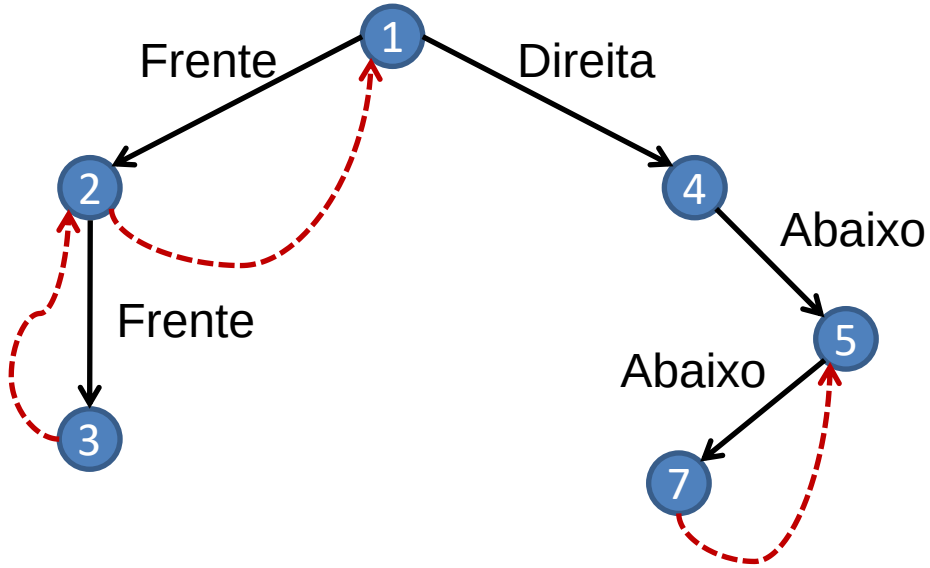
Backtracking - Labirinto

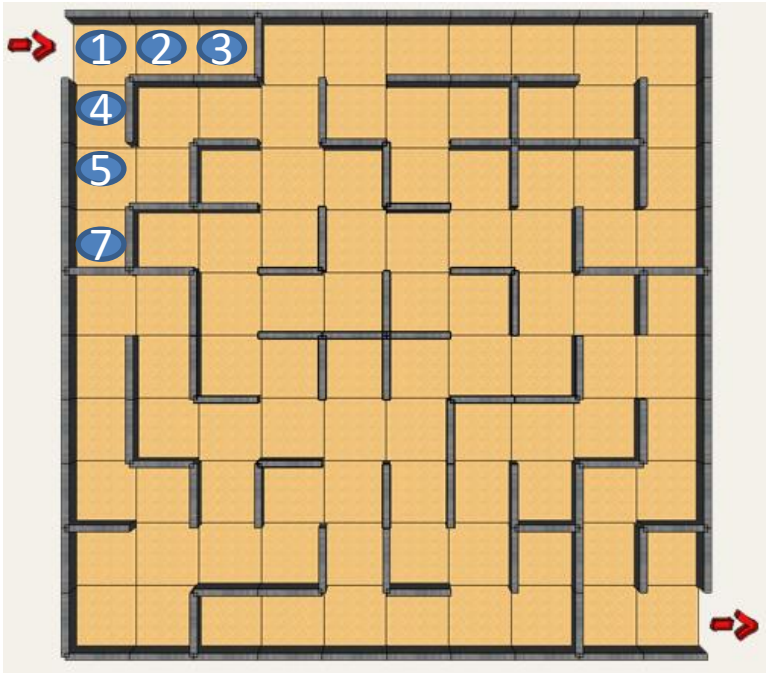




Backtracking

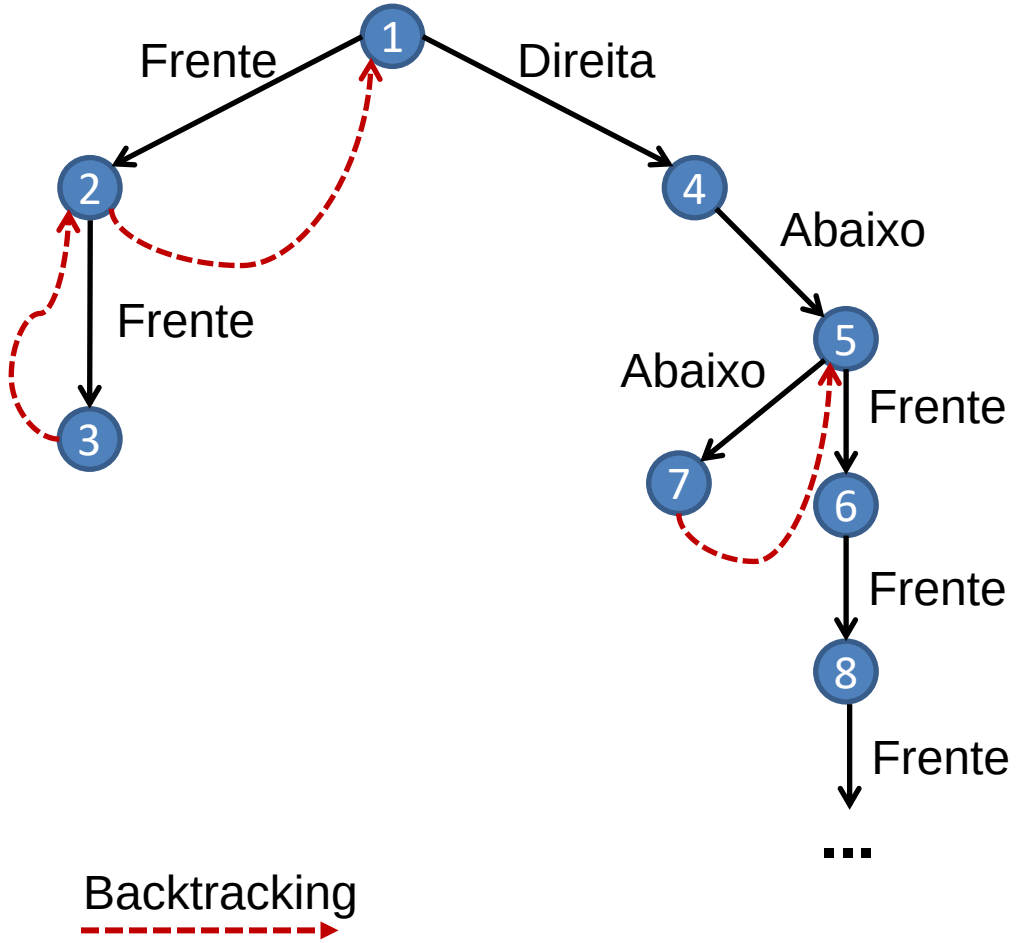
Backtracking - Labirinto

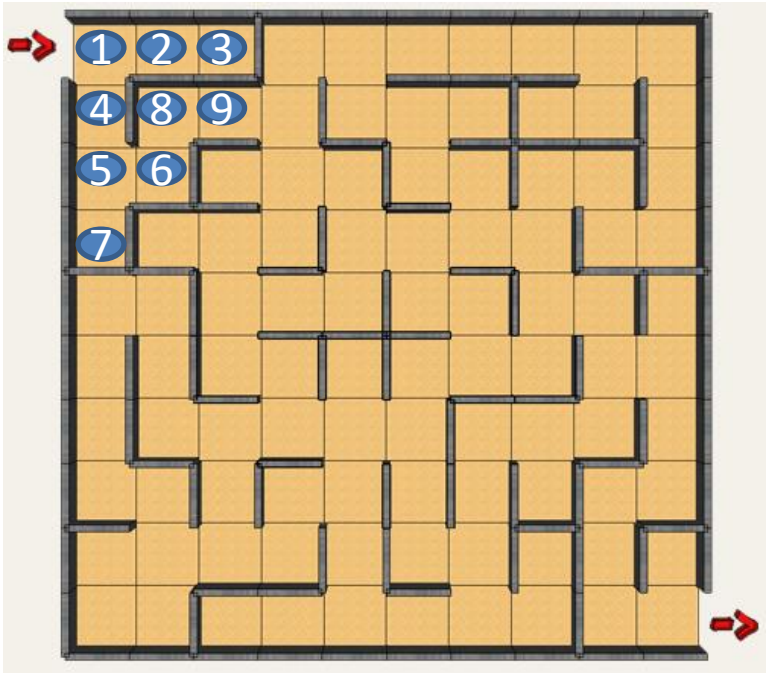




Backtracking

Backtracking - Labirinto





Backtracking

- **Passos**
  - Escolher uma operação plausível.
  - Executar uma operação com os dados do problema.
  - Se a solução não foi alcançada, repita o processo até que se atinja a solução ou se evidencie a insolubilidade do problema.

Backtracking

- Construir uma árvore com as escolhas a fazer: *árvore de estados*

Backtracking

- Pode ser estratégia em profundidade (DFS) ou largura (BFS). Em geral, é DFS

- A raiz da árvore representa o estado inicial do problema.

- Os nós do segundo nível representam a primeira escolha possível. Os nós do terceiro nível representam a segunda escolha possível. Etc...

Backtracking

- Pode ser estratégia em profundidade (DFS) ou largura (BFS). Em geral, é DFS

- A raiz da árvore representa o estado inicial do problema.

- Os nós do segundo nível representam a primeira escolha possível. Os nós do terceiro nível representam a segunda escolha possível. Etc...

Backtracking

- Nó ***promissor***: corresponde a uma solução parcial que ainda pode levar a uma solução completa.

  Neste caso, um filho do nó é gerado.

- Nó ***não-promissor*** : corresponde a uma solução parcial, mas não existem novas escolhas possíveis para completar a solução.

  Neste caso, o algoritmo volta atrás na árvore (*backtrack*).

Backtracking

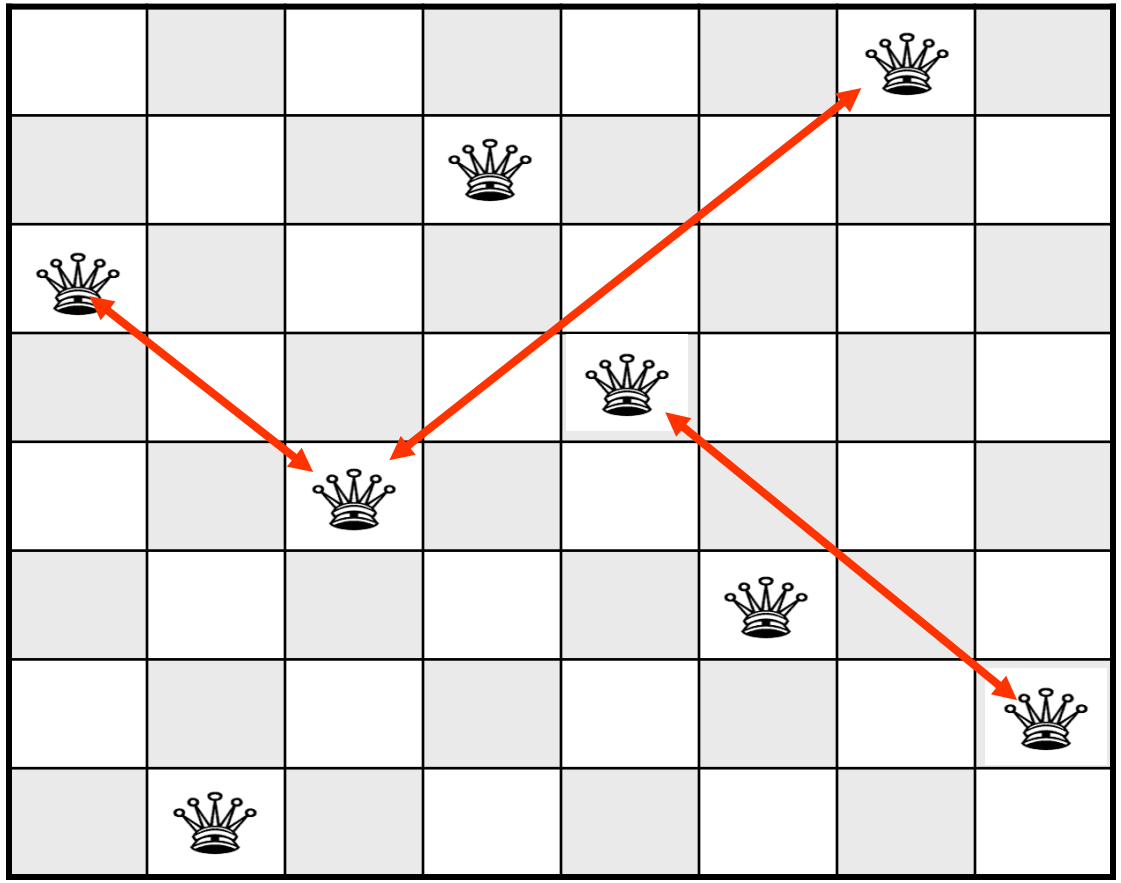
- Finalmente, o algoritmo encontra uma solução ou, após verificar todas as possibilidades, informa que tal solução não existe.

Backtracking - Resumo

- Utiliza grafo implícito (árvore) para enumeração.
- Em sua forma básica, se assemelha a uma busca em profundidade em um grafo dirigido.
- A ideia é interromper a busca quando o vetor de enumeração se torna inviável.
- O algoritmo encontra soluções parciais, as quais limitam a região do espaço de busca em que uma solução pode ser encontrada.

O Problema das *n* Rainhas

Alocar *n* rainhas em um tabuleiro *n* x *n* sem conflitos



3 conflitos

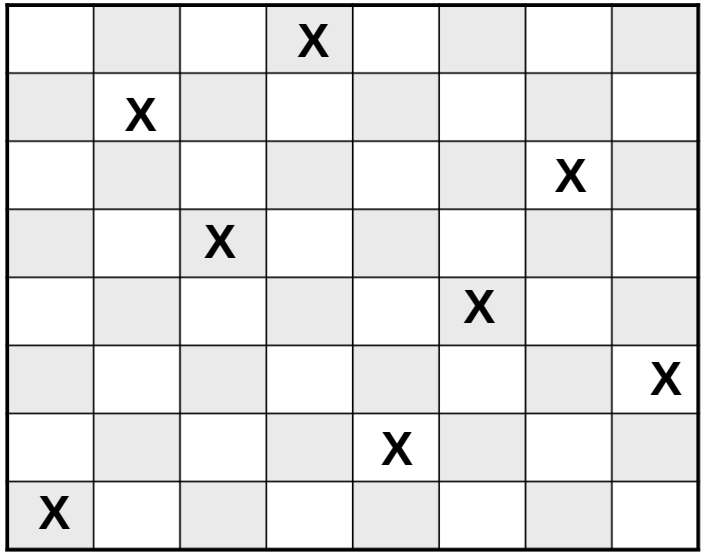
Backtracking – 8 Rainhas

Representação de uma solução: *permutação das colunas*

Vantagem: elimina preocupação com conflitos em linhas e em colunas.

4	2	7	3	6	8	5	1

- V[i] = j; i = linha; j = coluna.

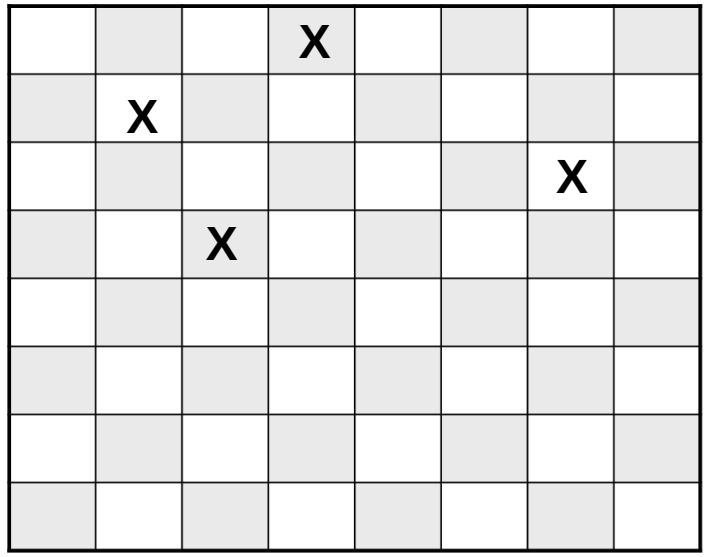


Backtracking – 8 Rainhas

- O vetor é dito ***k-promissor*** se até a posição k não há conflito entre as rainhas.

4	2	7	3				

vetor ***4-promissor***



Backtracking – 8 Rainhas

- N: conjunto de vetores k-promissores, com $0 \leq k \leq 8$.

- G = (N, A)

  grafo dirigido tal que $(U, V) \in A$ se e somente se existe um inteiro k, $0 \leq k \leq 8$, tal que:

  - U é k-promissor
  - V é (k+1)-promissor e
  - U[i] = V[i] para todo $i \in [1..k]$

Backtracking

sol[1..k] é k-promissora

col = {sol[i] | 1 ≤ i ≤ k} \\ colunas proibidas na linha i

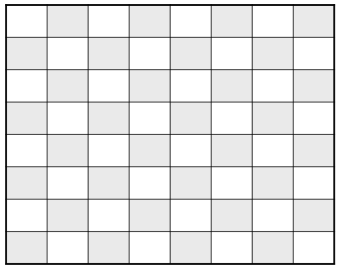
diag45 = {sol[i] – i + 1 | 1 ≤ i ≤ k} \\ inteiro para cálculo das diagonais 45º proibidas na linha i

diag135 = {sol[i] + i - 1 | 1 ≤ i ≤ k} \\ inteiro para cálculo das diagonais 135º proibidas na linha i

8Rainhas_Track (k, col, diag45, diag135)

```
se k = 8 então escreva(sol)
 senão \\ explora as extensões de sol que são (k+1)-promissoras
    para j ← 1 até 8 faça
        se j ∉ col e j – k ∉ diag45 e j+k ∉ diag135 então
            8Rainhas_Track(k+1,col∪{j}, diag45∪{j-k}, diag135∪{j+k})
```

Backtracking – 8 rainhas



8Rainhas_Track (k, col, diag45, diag135)

se k = 8 **então escreva**(sol)
senão \\ explora as extensões de sol que são (k+1)-promissoras
  para j ← 1 **até** 8 **faça**
    se j ∉ col **e** j – k ∉ diag45 **e** j+k ∉ diag135 **então**
      8Rainhas_Track(k+1,col∪{j}, diag45∪{j-k}, diag135∪{j+k})

k = 0

Sol = [0,0,0,0,0,0,0,0]

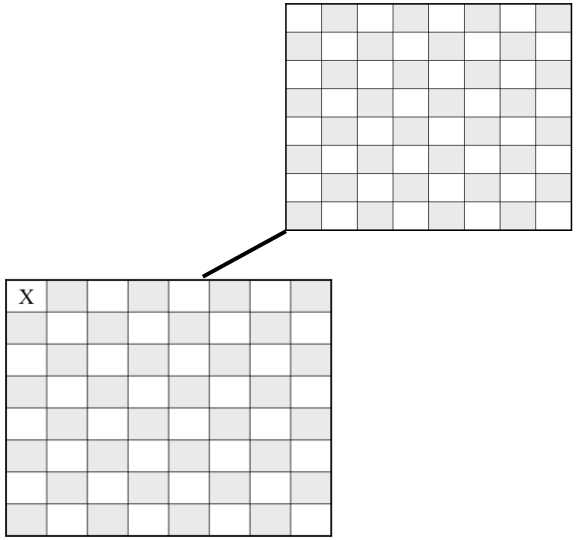
col = [0,0,0,0,0,0,0,0]

diag45 = [0,0,0,0,0,0,0,0]

diag135 = [0,0,0,0,0,0,0,0]

j = 1 **OK**

Backtracking – 8 rainhas



8Rainhas_Track (k, col, diag45, diag135)

se k = 8 **então escreva**(sol)
senão \\ explora as extensões de sol que são (k+1)-promissoras
  para j ← 1 **até** 8 **faça**
    se j ∉ col **e** j – k ∉ diag45 **e** j+k ∉ diag135 **então**
      8Rainhas_Track(k+1,col∪{j}, diag45∪{j-k}, diag135∪{j+k})

k = 1

Sol = [1,0,0,0,0,0,0,0]

col = [1,0,0,0,0,0,0,0]

diag45 = [1,0,0,0,0,0,0,0]

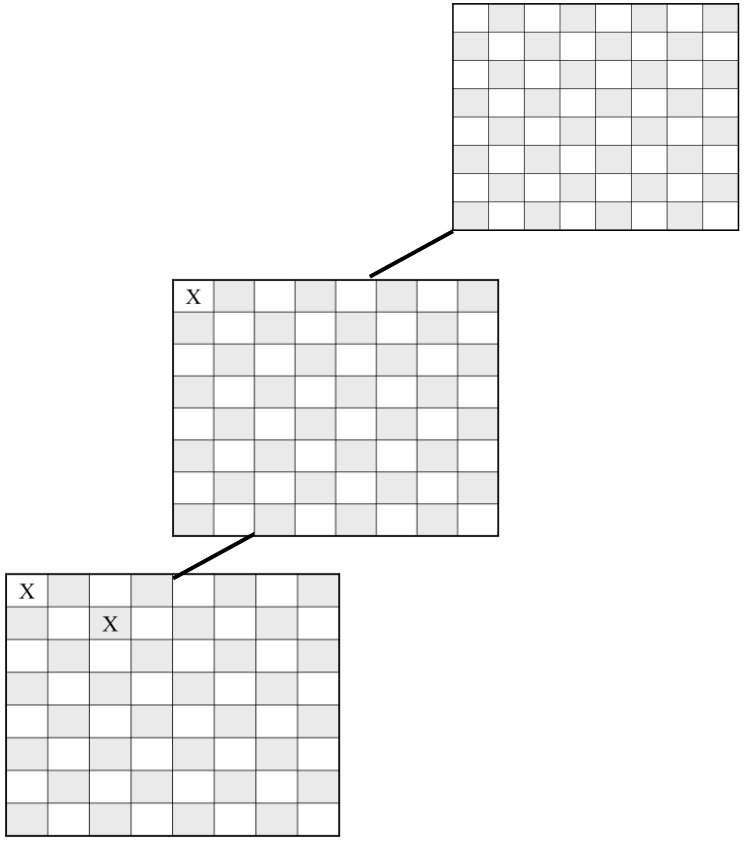
diag135 = [1,0,0,0,0,0,0,0]

j = 1 X (coluna)

j = 2 X (diag135)

j = 3 **OK**

Backtracking – 8 rainhas



8Rainhas_Track (k, col, diag45, diag135)

se k = 8 **então escreva**(sol)
senão \\ explora as extensões de sol que são (k+1)-promissoras
  para j ← 1 **até** 8 **faça**
    se j ∉ col **e** j – k ∉ diag45 **e** j+k ∉ diag135 **então**
      8Rainhas_Track(k+1,col∪{j}, diag45∪{j-k}, diag135∪{j+k})

k = 2

Sol = [1,3,0,0,0,0,0,0]

col = [1,3,0,0,0,0,0,0]

diag45 = [1,0,0,0,0,0,0,0]

diag135 = [1,2,0,0,0,0,0,0]

j = 1 X (coluna)

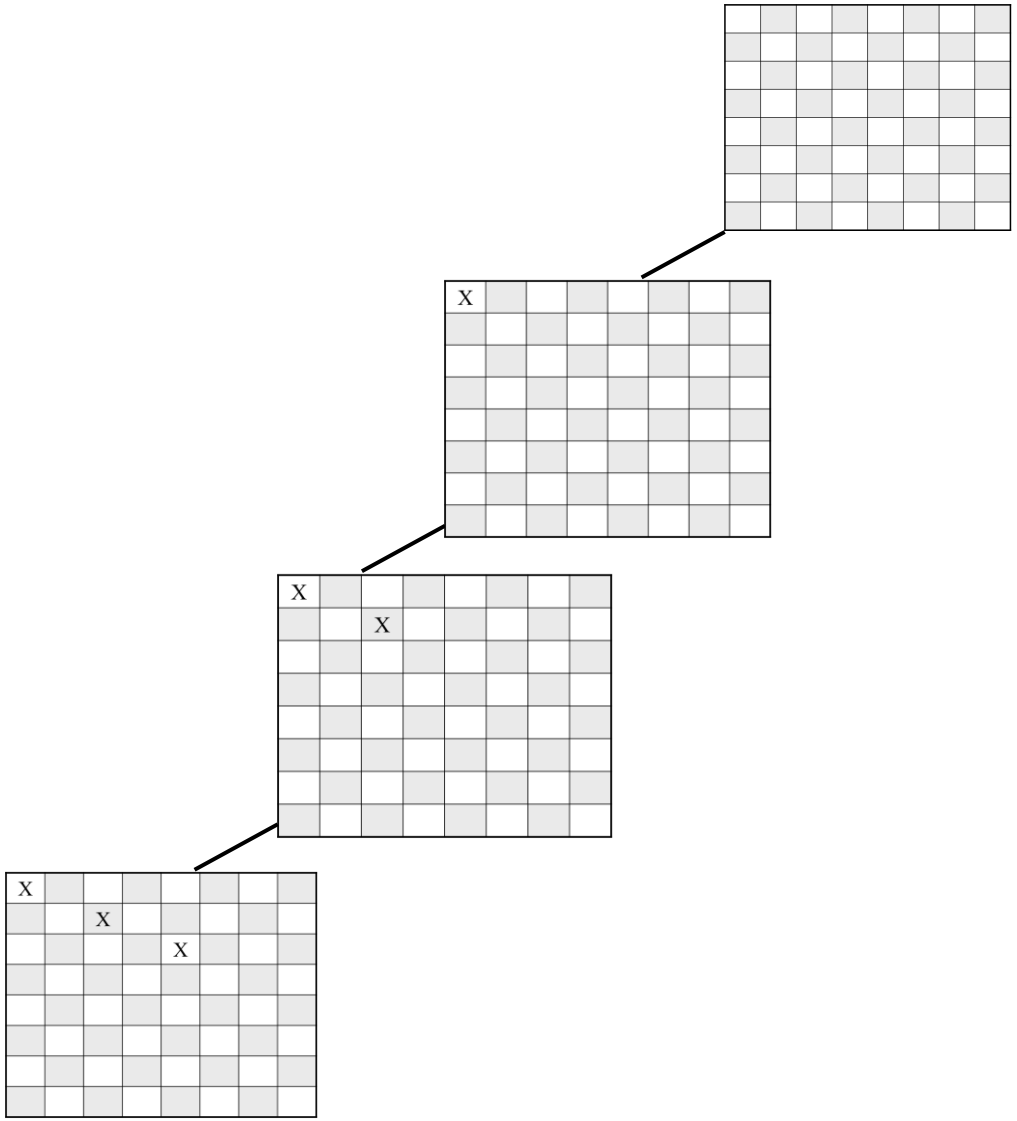
j = 2 X (diag45)

j = 3 X (coluna)

j = 4 X (diag135)

j = 5 **OK**

Backtracking – 8 rainhas




```
8Rainhas_Track (k, col, diag45, diag135)
se k = 8 então escreva(sol)
 senão \\ explora as extensões de sol que são (k+1)-promissoras
    para j ← 1 até 8 faça
       se j ∉ col e j – k ∉ diag45 e j+k ∉ diag135 então
          8Rainhas_Track(k+1,col∪{j}, diag45∪{j-k}, diag135∪{j+k})
```

k = 3

Sol = [1,3,5,0,0,0,0,0]

col = [1,3,5,0,0,0,0,0]

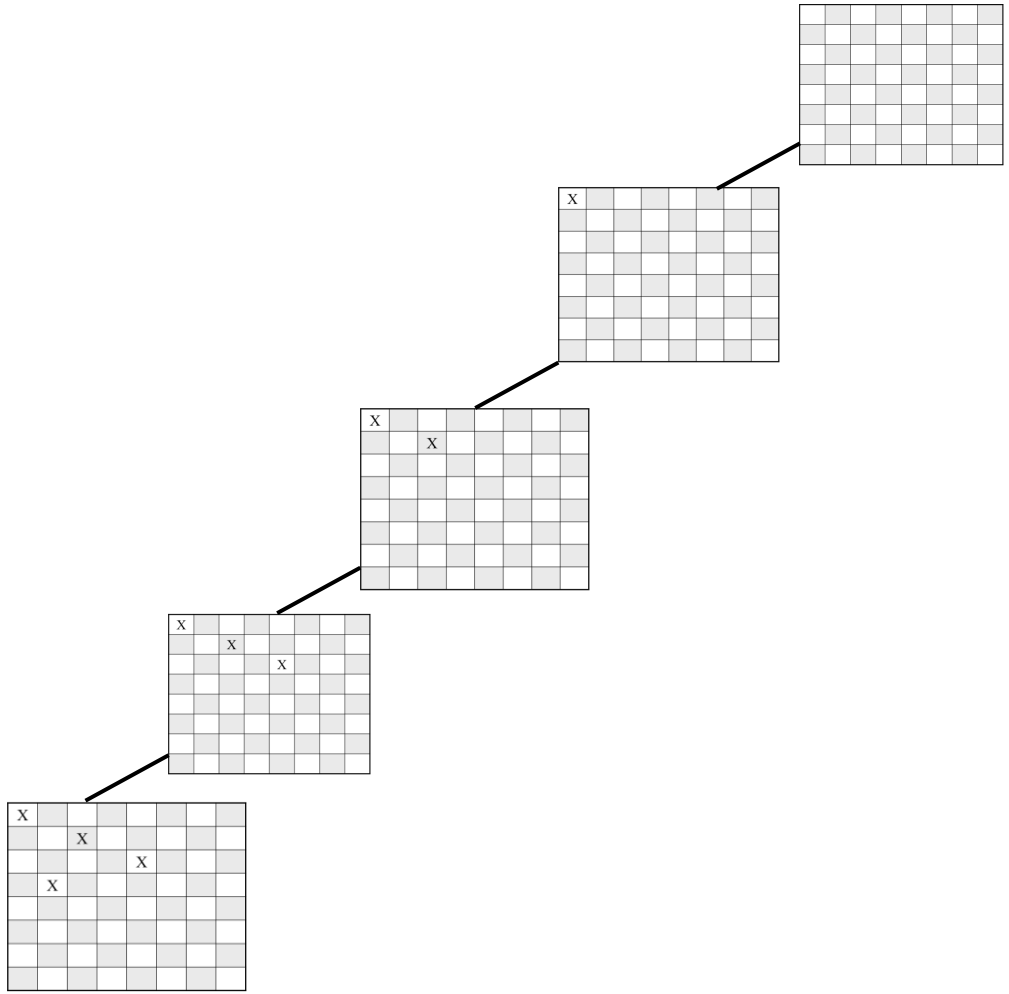
diag45 = [1,3,0,0,0,0,0,0]

diag135 = [1,2,7,0,0,0,0,0]

j = 1 X (coluna)

j = 2 **OK**

Backtracking – 8 rainhas



```
8Rainhas_Track (k, col, diag45, diag135)
se k = 8 então escreva(sol)
 senão \\ explora as extensões de sol que são (k+1)-promissoras
    para j ← 1 até 8 faça
       se j ∉ col e j – k ∉ diag45 e j+k ∉ diag135 então
          8Rainhas_Track(k+1,col∪{j}, diag45∪{j-k}, diag135∪{j+k})
```

k = 4

Sol = [1,3,5,2,0,0,0,0]

col = [1,3,5,2,0,0,0,0]

diag45 = [1,3,0,0,0,0,0,0]

diag135 = [1,2,7,5,0,0,0,0]

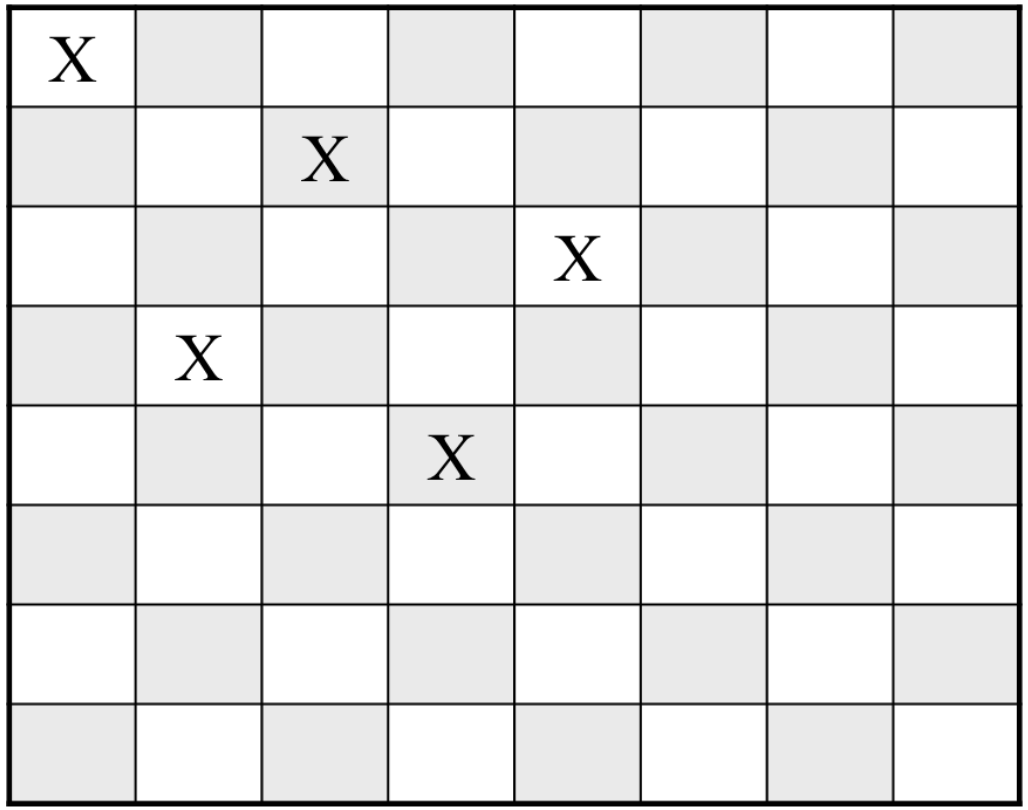
j = 1 X (coluna)

j = 2 X (coluna)

j = 3 X (diag135)

j = 4 **OK**

Backtracking – 8 rainhas



```
8Rainhas_Track (k, col, diag45, diag135)
se k = 8 então escreva(sol)
 senão \\ explora as extensões de sol que são (k+1)-promissoras
    para j ← 1 até 8 faça
       se j ∉ col e j – k ∉ diag45 e j+k ∉ diag135 então
          8Rainhas_Track(k+1,col∪{j}, diag45∪{j-k}, diag135∪{j+k})
```

k = 5

Sol = [1,3,5,2,4,0,0,0]

col = [1,3,5,2,4,0,0,0]

diag45 = [1,3,0,0,0,0,0,0]

diag135 = [1,2,7,5,8,0,0,0]

j = 1 X (coluna)

j = 2 X (coluna)

j = 3 X (diag135)

j = 4 X (coluna)

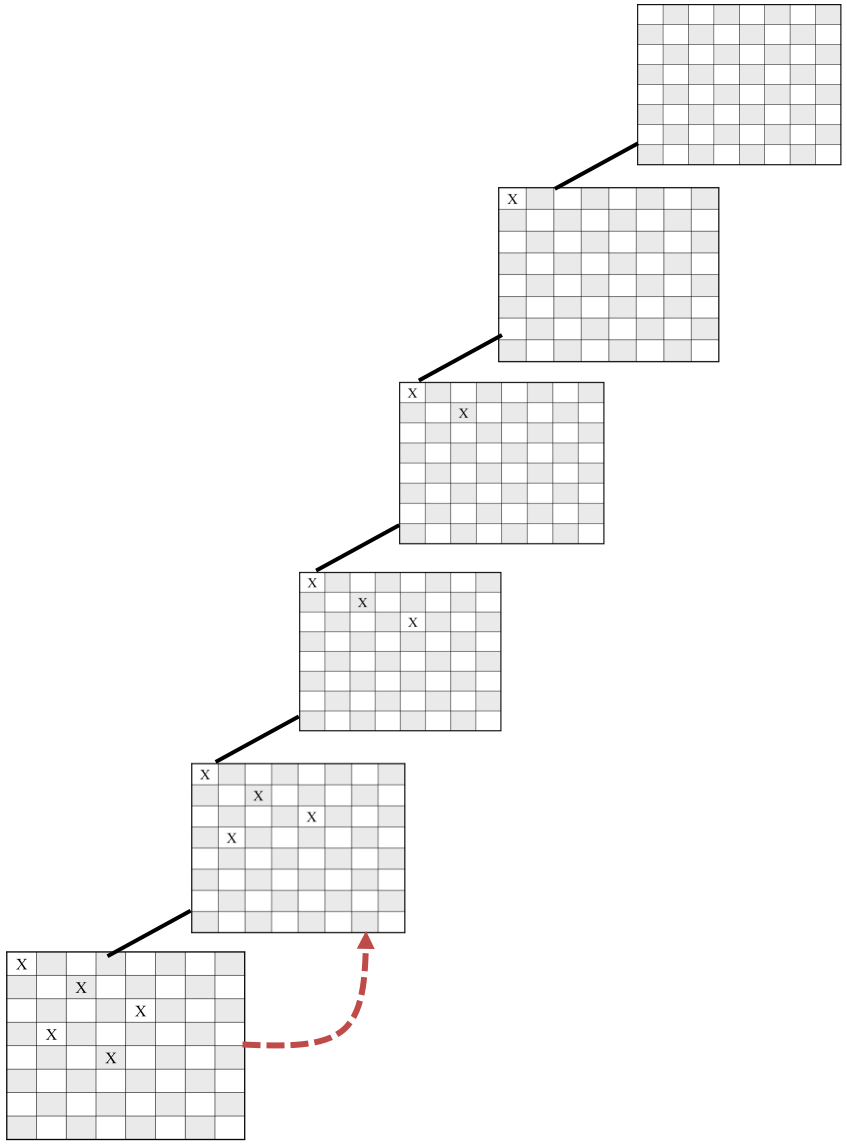
j = 5 X (coluna)

j = 6 X (diag135)

j = 7 X (diag135)

j = 8 X (diag135)

Backtracking – 8 rainhas



```
8Rainhas_Track (k, col, diag45, diag135)
se k = 8 então escreva(sol)
 senão \\ explora as extensões de sol que são (k+1)-promissoras
    para j ← 1 até 8 faça
       se j ∉ col e j – k ∉ diag45 e j+k ∉ diag135 então
          8Rainhas_Track(k+1,col∪{j}, diag45∪{j-k}, diag135∪{j+k})
```

k = 5

Sol = [1,3,5,2,4,0,0,0]

col = [1,3,5,2,4,0,0,0]

diag45 = [1,3,0,0,0,0,0,0]

diag135 = [1,2,7,5,8,0,0,0]

j = 1 X (coluna)

j = 2 X (coluna)

j = 3 X (diag135)

j = 4 X (coluna)

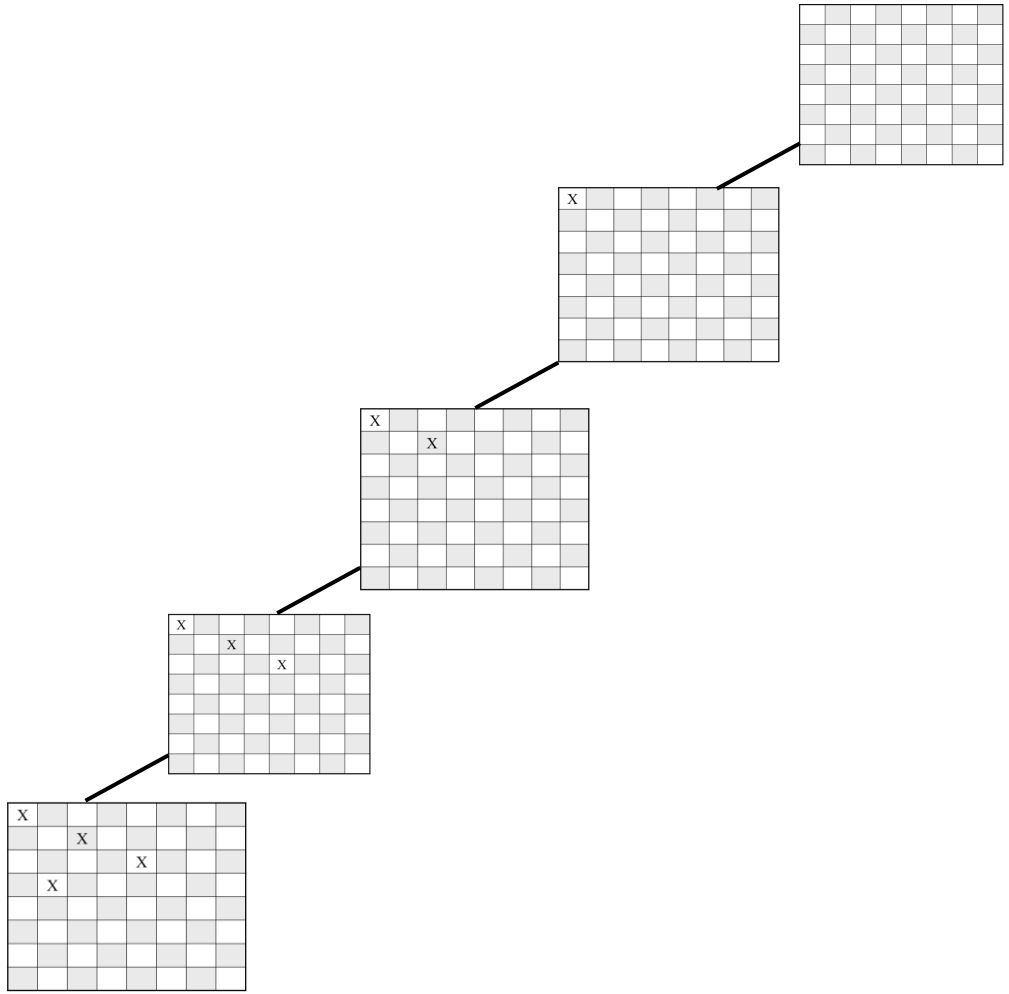
j = 5 X (coluna)

j = 6 X (diag135)

j = 7 X (diag135)

j = 8 X (diag135)

Backtracking – 8 rainhas



8Rainhas_Track (k, col, diag45, diag135)

se k = 8 **então escreva**(sol)
senão \\ explora as extensões de sol que são (k+1)-promissoras
  para j ← 1 **até** 8 **faça**
    se j ∉ col **e** j – k ∉ diag45 **e** j+k ∉ diag135 **então**
      8Rainhas_Track(k+1,col∪{j}, diag45∪{j-k}, diag135∪{j+k})

k = 4

Sol = [1,3,5,2,0,0,0,0]

col = [1,3,5,2,0,0,0,0]

diag45 = [1,3,0,0,0,0,0,0]

diag135 = [1,2,7,5,0,0,0,0]

j = 1 X (coluna)

j = 2 X (coluna)

j = 3 X (diag135)

j = 4 **Backtrack**

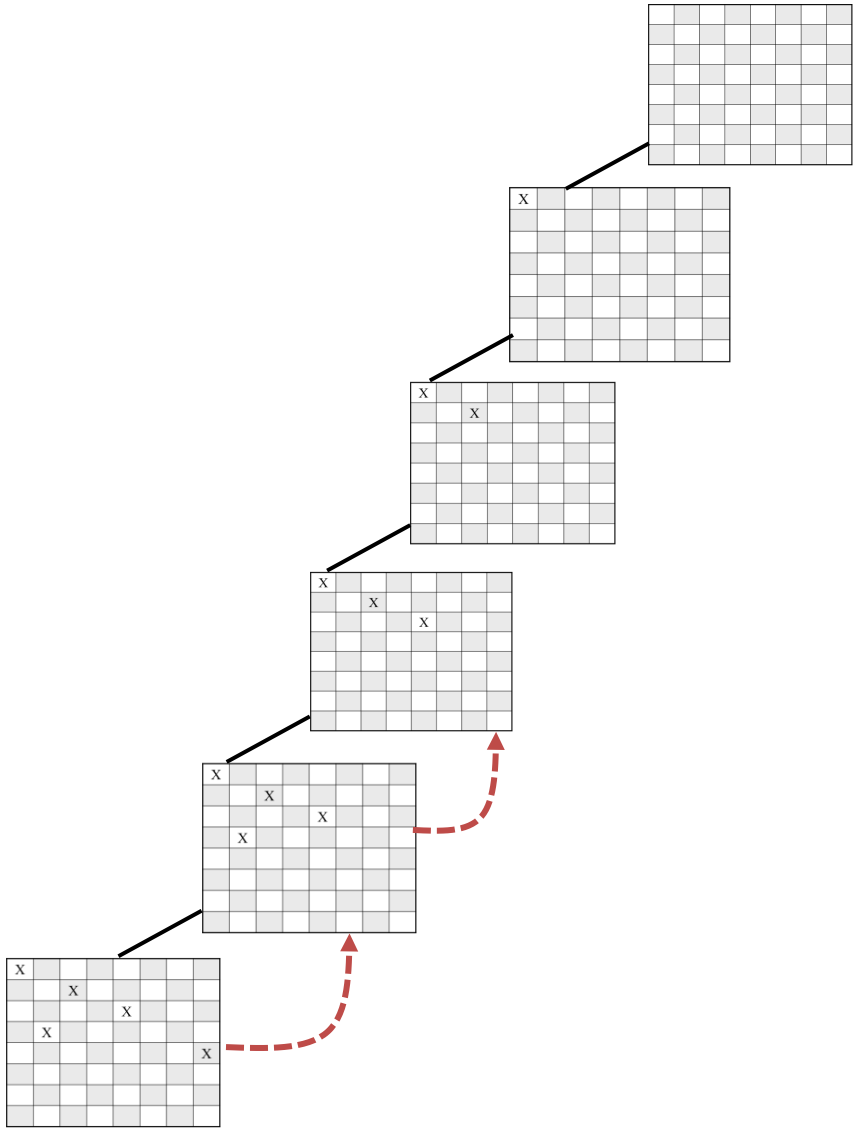
j = 5 X (coluna)

j = 6 X (diag135)

j = 7 X (diag135)

j = 8 **OK**

Backtracking – 8 rainhas



```
8Rainhas_Track (k, col, diag45, diag135)
se k = 8 então escreva(sol)
 senão \\ explora as extensões de sol que são (k+1)-promissoras
    para j ← 1 até 8 faça
       se j ∉ col e j – k ∉ diag45 e j+k ∉ diag135 então
          8Rainhas_Track(k+1,col∪{j}, diag45∪{j-k}, diag135∪{j+k})
```

k = 5

Sol = [1,3,5,2,8,0,0,0]

col = [1,3,5,2,8,0,0,0]

diag45 = [1,3,4,0,0,0,0,0]

diag135 = [1,2,7,5,0,0,0,0]

j = 1 X (coluna)

j = 2 X (coluna)

j = 3 X (diag135)

j = 4 X (diag135)

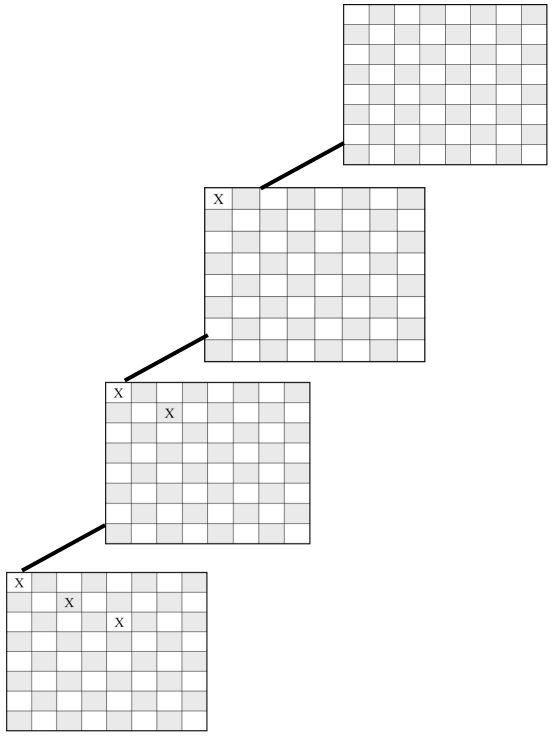
j = 5 X (coluna)

j = 6 X (diag135)

j = 7 X (diag135)

j = 8 X (coluna)

Backtracking – 8 rainhas



```
8Rainhas_Track (k, col, diag45, diag135)
se k = 8 então escreva(sol)
 senão \\ explora as extensões de sol que são (k+1)-promissoras
    para j ← 1 até 8 faça
       se j ∉ col e j – k ∉ diag45 e j+k ∉ diag135 então
          8Rainhas_Track(k+1,col∪{j}, diag45∪{j-k}, diag135∪{j+k})
```

k = 3

Sol = [1,3,5,0,0,0,0,0]

col = [1,3,5,0,0,0,0,0]

diag45 = [1,3,0,0,0,0,0,0]

diag135 = [1,2,7,0,0,0,0,0]

j = 1 X (coluna)

j = 2 Backtrack

j = 3 X (coluna)

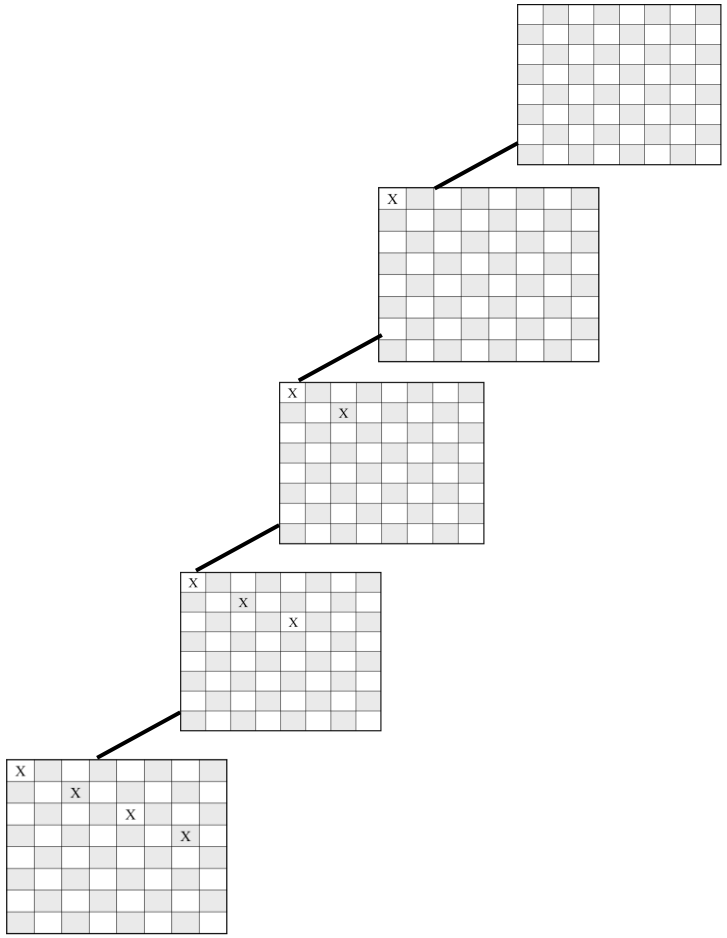
j = 4 X (diag135)

j = 5 X (coluna

j = 6 X (diag135)

j = 7 **OK**

Backtracking – 8 rainhas



```
8Rainhas_Track (k, col, diag45, diag135)
se k = 8 então escreva(sol)
 senão \\ explora as extensões de sol que são (k+1)-promissoras
    para j ← 1 até 8 faça
       se j ∉ col e j – k ∉ diag45 e j+k ∉ diag135 então
          8Rainhas_Track(k+1,col∪{j}, diag45∪{j-k}, diag135∪{j+k})
```

k = 4

Sol = [1,3,5,7,0,0,0,0]

col = [1,3,5,7,0,0,0,0]

diag45 = [1,3,4,0,0,0,0,0]

diag135 = [1,2,7,0,0,0,0,0]

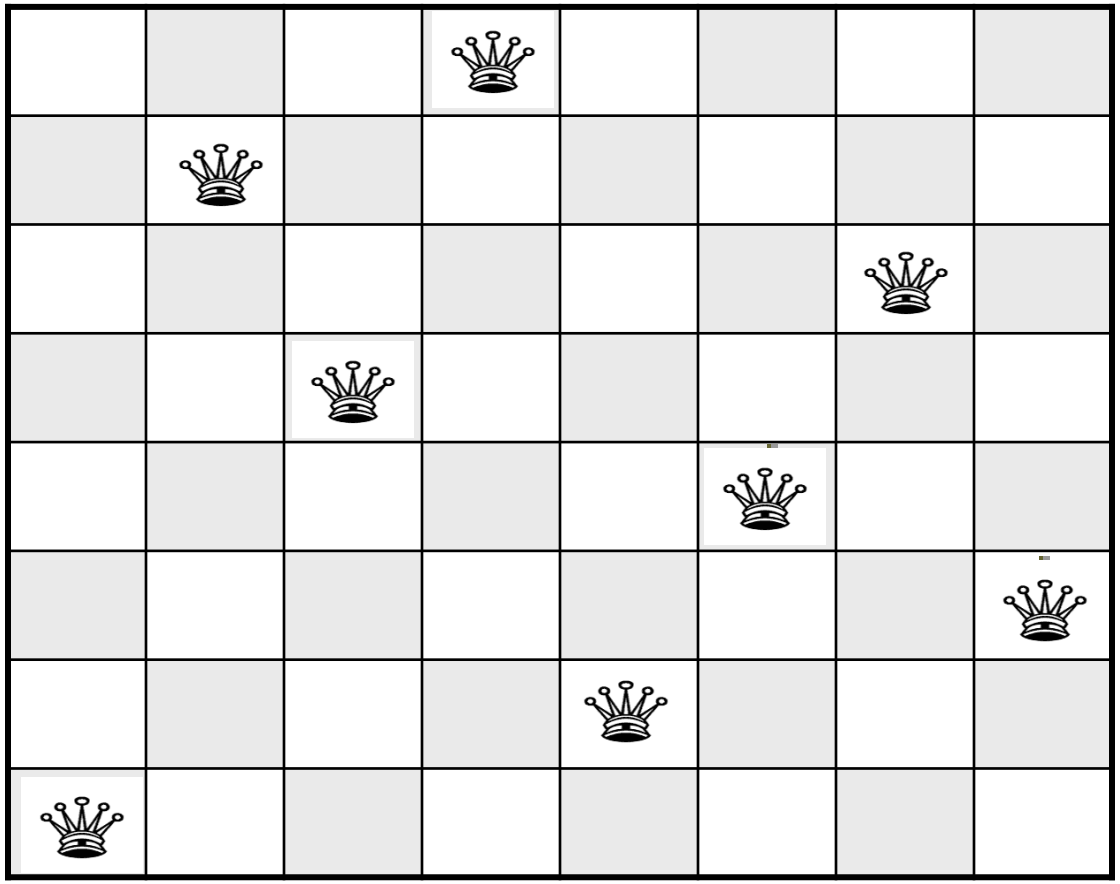
j = 1 X (coluna)

j = 2 **OK**

...

O Problema das *n* Rainhas

Uma das 92 Soluções (12 padrões que são rotacionados)



Bracktracking – Quadro geral

Backtrack (v[1..k])

```
{v é um vetor k-promissor}
se v é uma solução então escreva(v)
senão
    para cada vetor w (k+1)-promissor tal que w[1..k] = v[1..k] faça
        Backtrack (w[1..k+1])
```

Problema da Mochila

$$z = \max \sum_{j=1}^{n} v_j x_j$$

Sujeito a :

$$\sum_{j=1}^{n} w_j x_j \leq W$$

$$v_j, w_j > 0 \text{ e inteiro, } j = 1,2,3,..., n$$

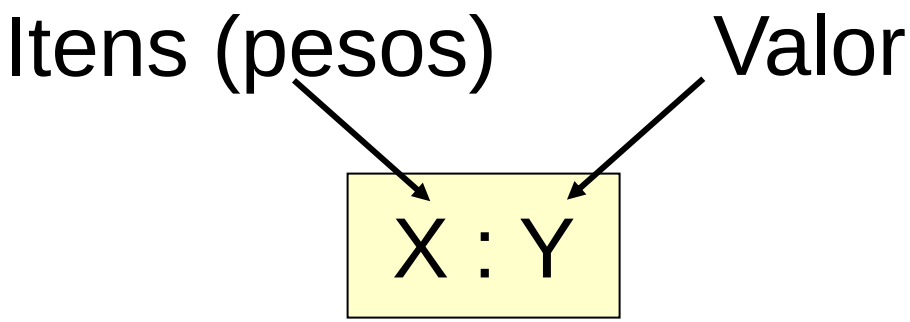
$$x_j \geq 0 \text{ e inteiro, } j = 1,2,3,..., n$$

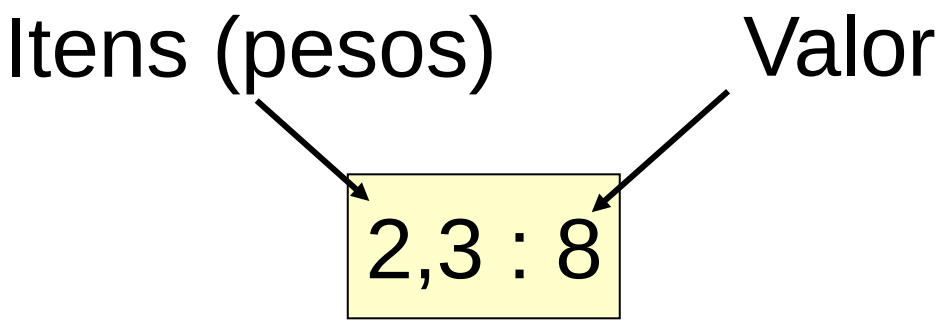
Backtracking – Problema da Mochila

- Exemplo:

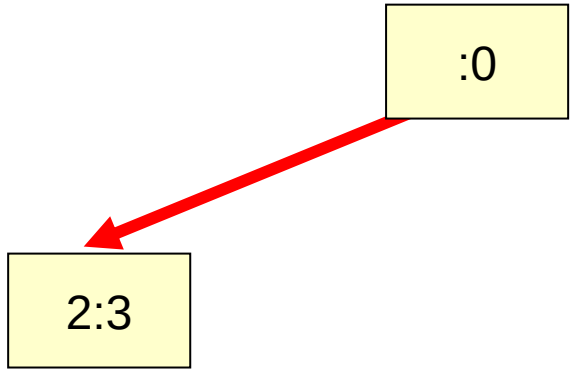
  - Item: 1 2 3 4
  - Pesos: 2 3 4 5
  - Valores: 3 5 6 10
  - Peso máximo: 8

Backtracking – Problema da Mochila



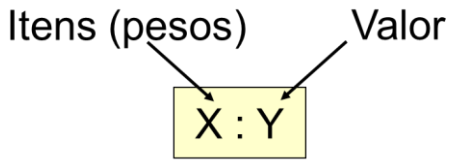


Backtracking

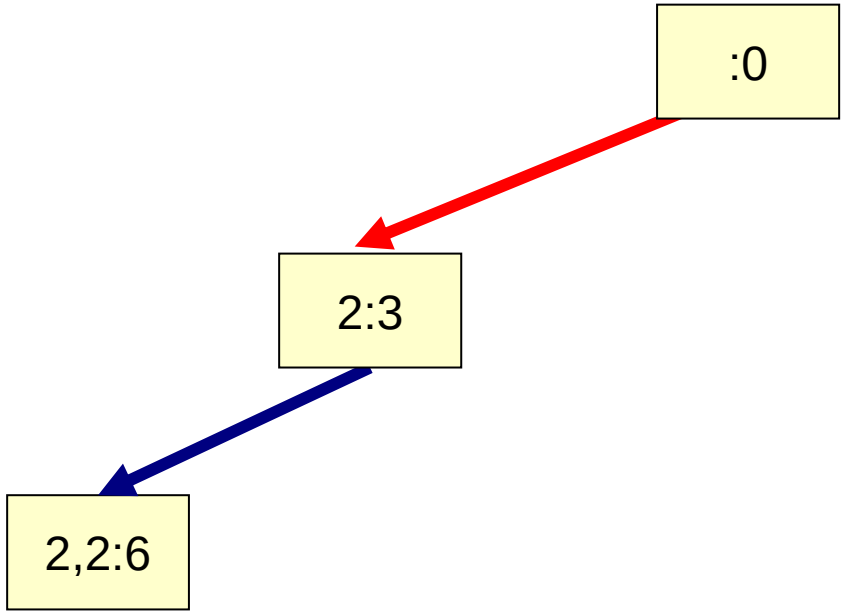


item	1	2	3	4
peso	2	3	4	5
valor	3	5	6	10

Peso máximo: 8

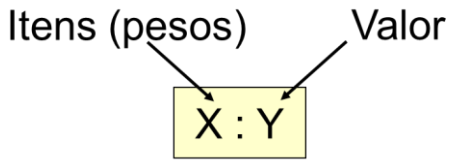


Backtracking

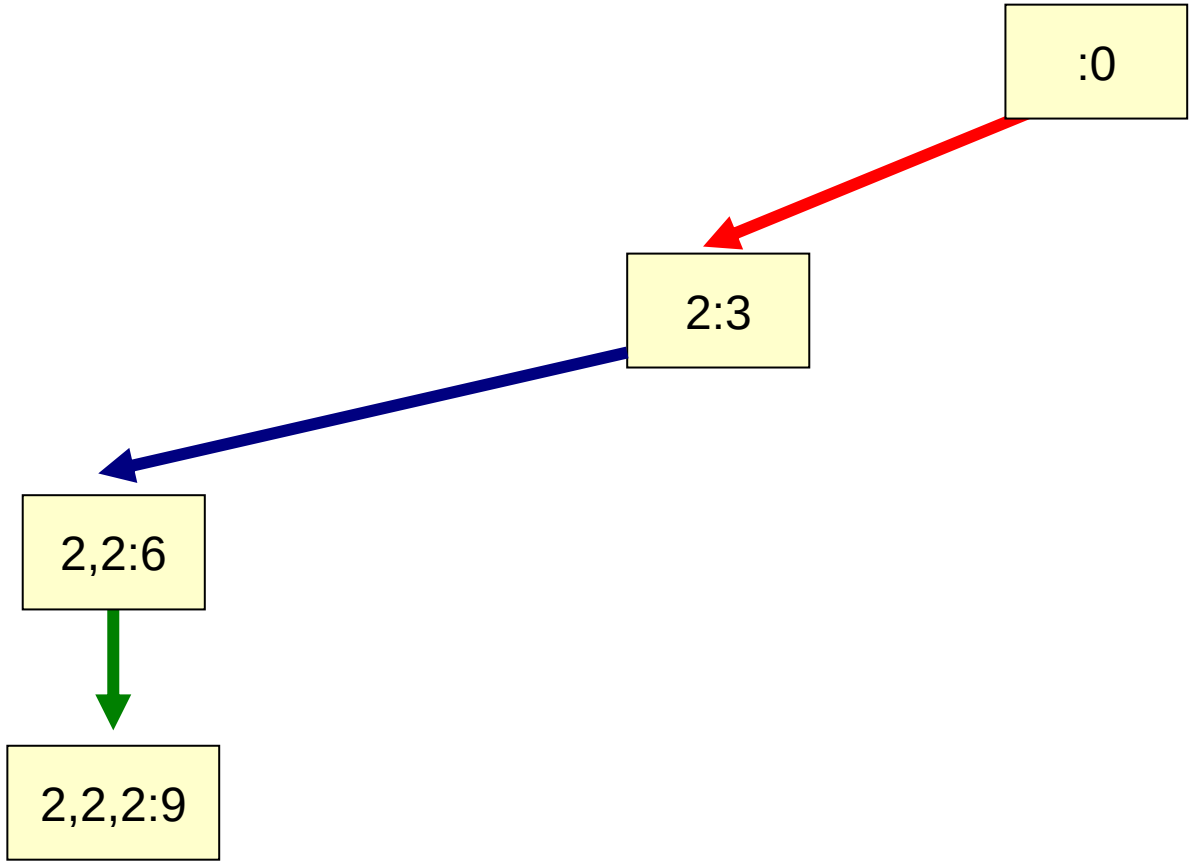


item	1	2	3	4
peso	2	3	4	5
valor	3	5	6	10

Peso máximo: 8

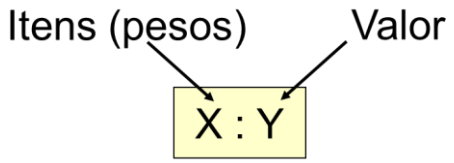


Backtracking

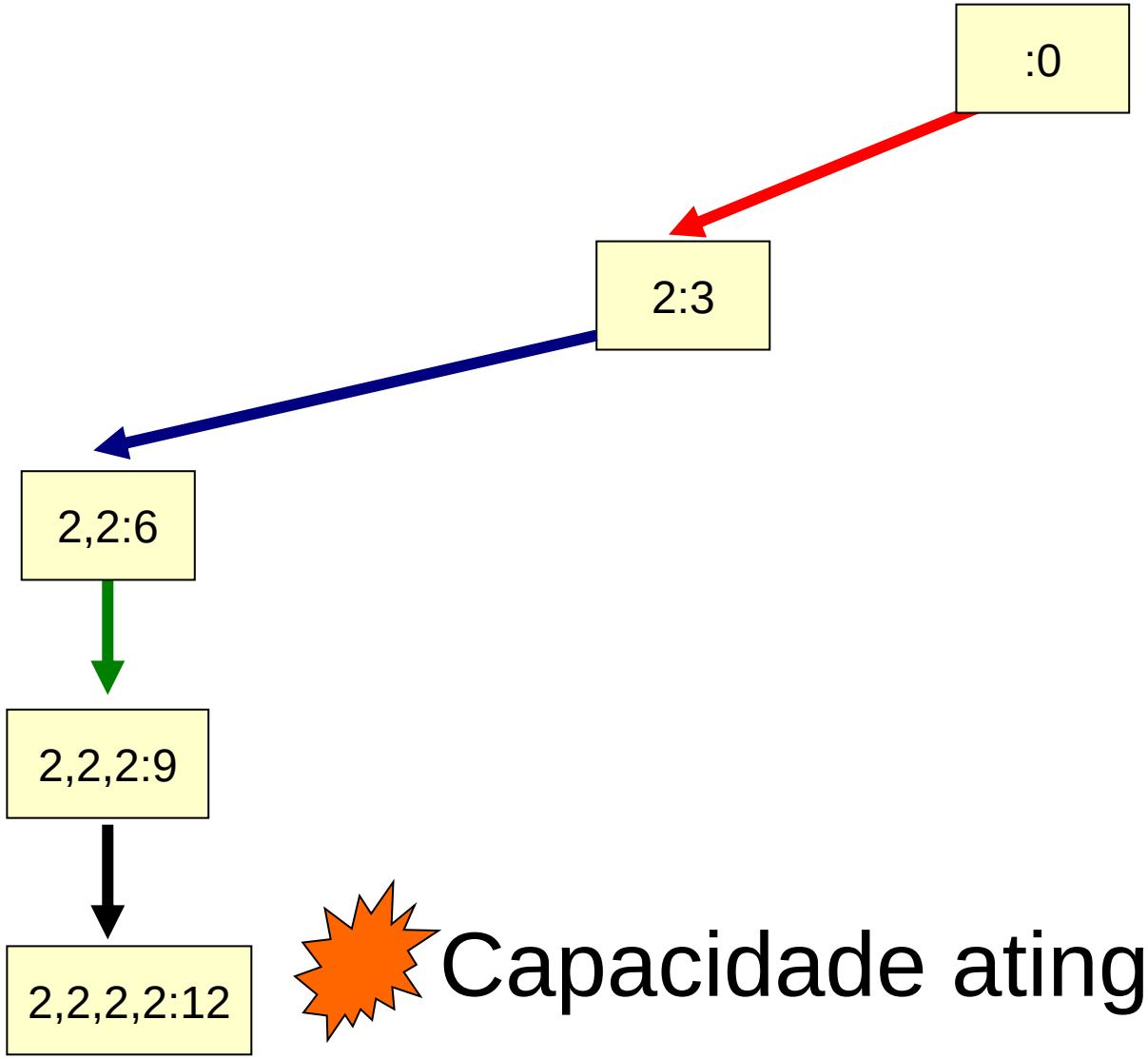


item	1	2	3	4
peso	2	3	4	5
valor	3	5	6	10

Peso máximo: 8

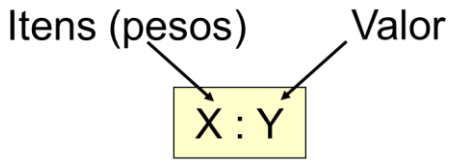


Backtracking

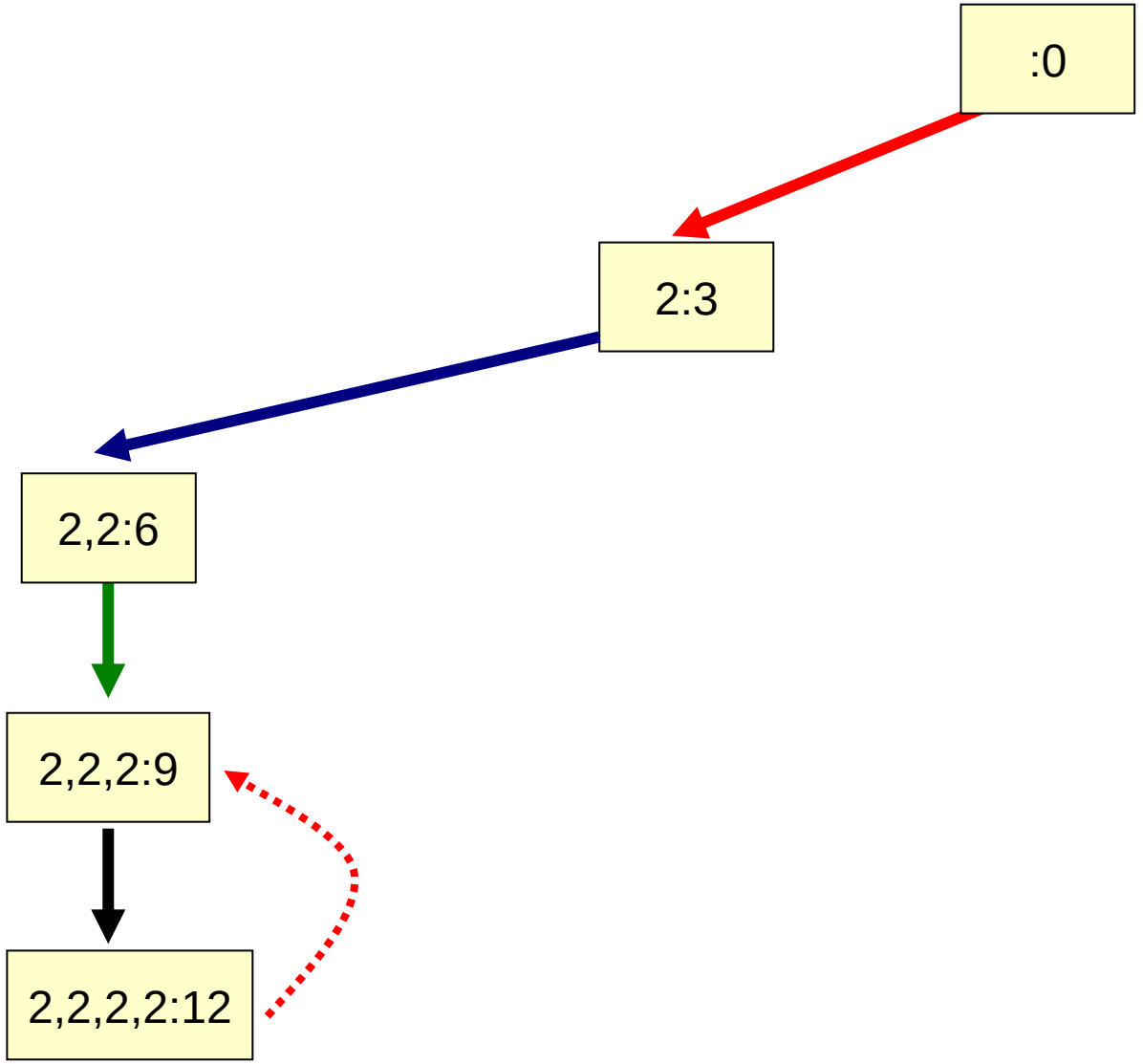


item	1	2	3	4
peso	2	3	4	5
valor	3	5	6	10

Peso máximo: 8

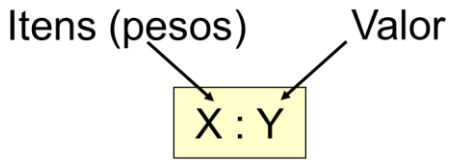


Backtracking

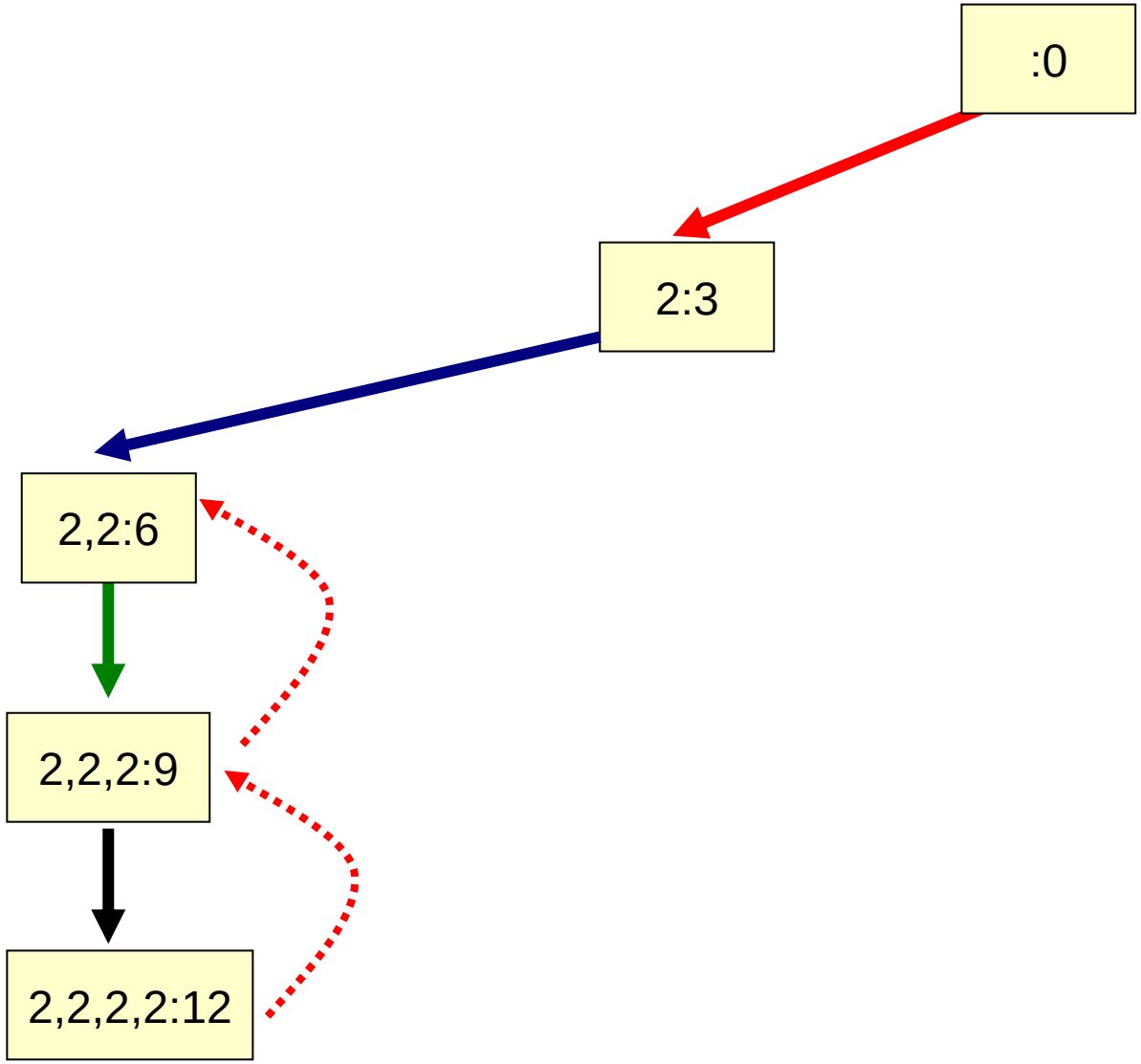


item	1	2	3	4
peso	2	3	4	5
valor	3	5	6	10

Peso máximo: 8

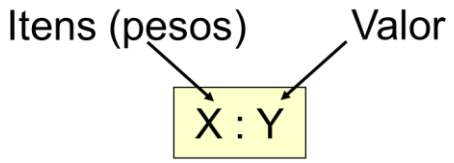


Backtracking

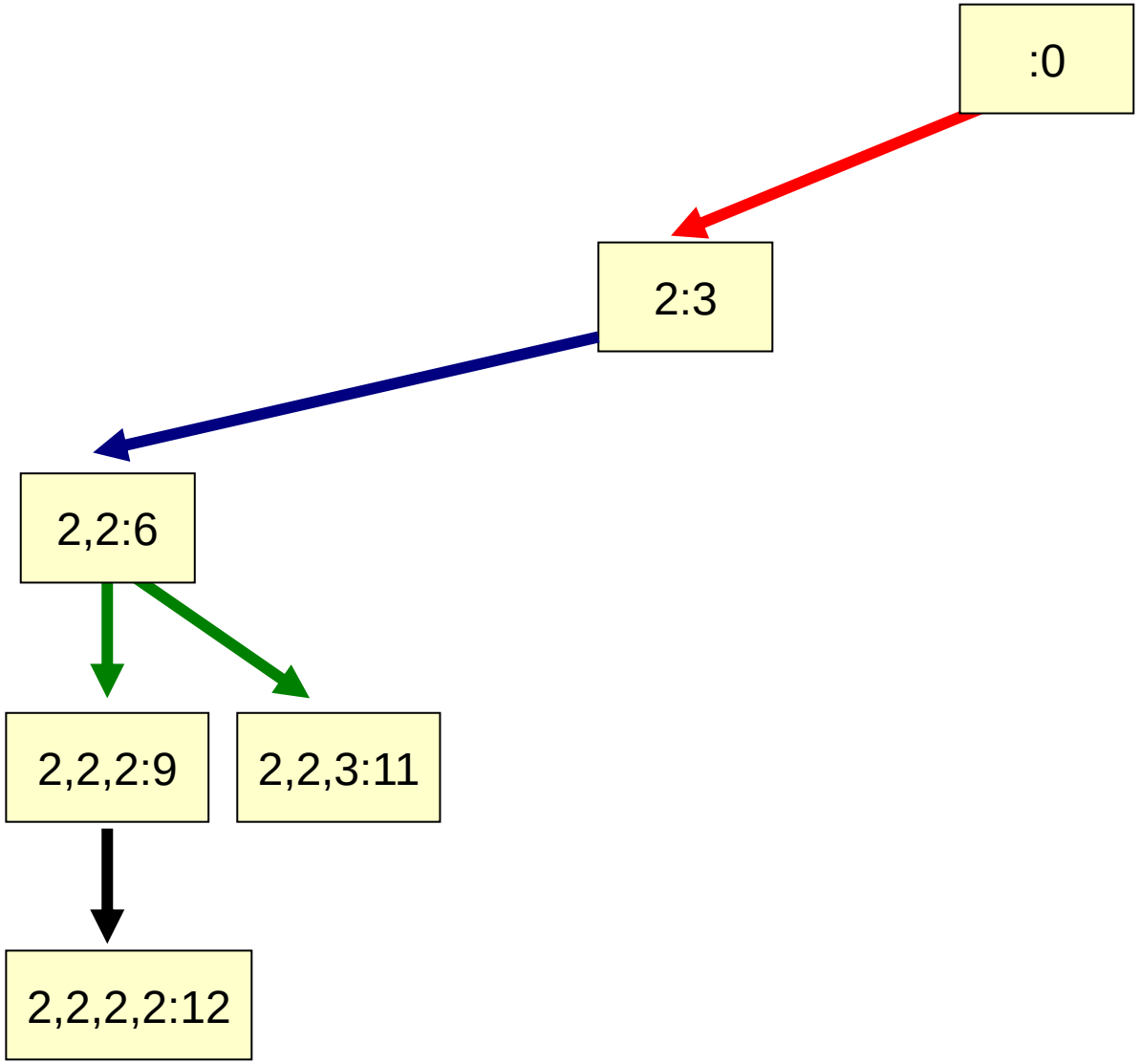


item	1	2	3	4
peso	2	3	4	5
valor	3	5	6	10

Peso máximo: 8

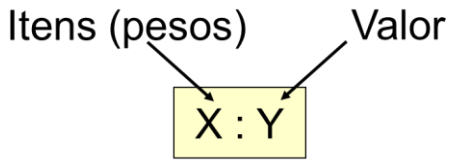


Backtracking

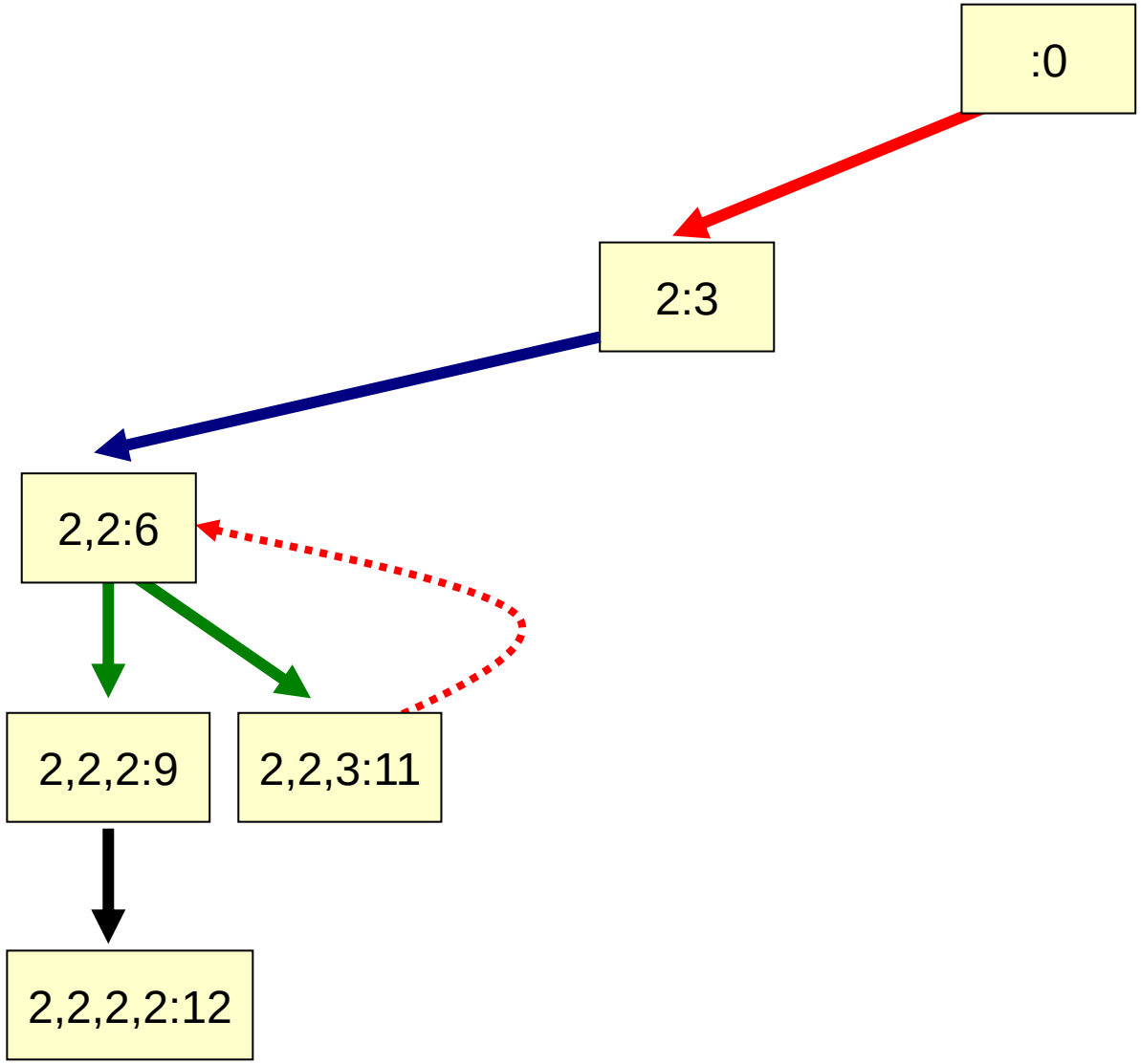


item	1	2	3	4
peso	2	3	4	5
valor	3	5	6	10

Peso máximo: 8

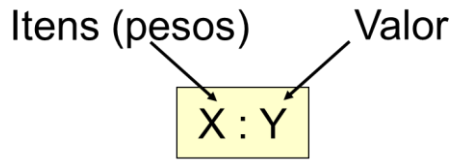


Backtracking

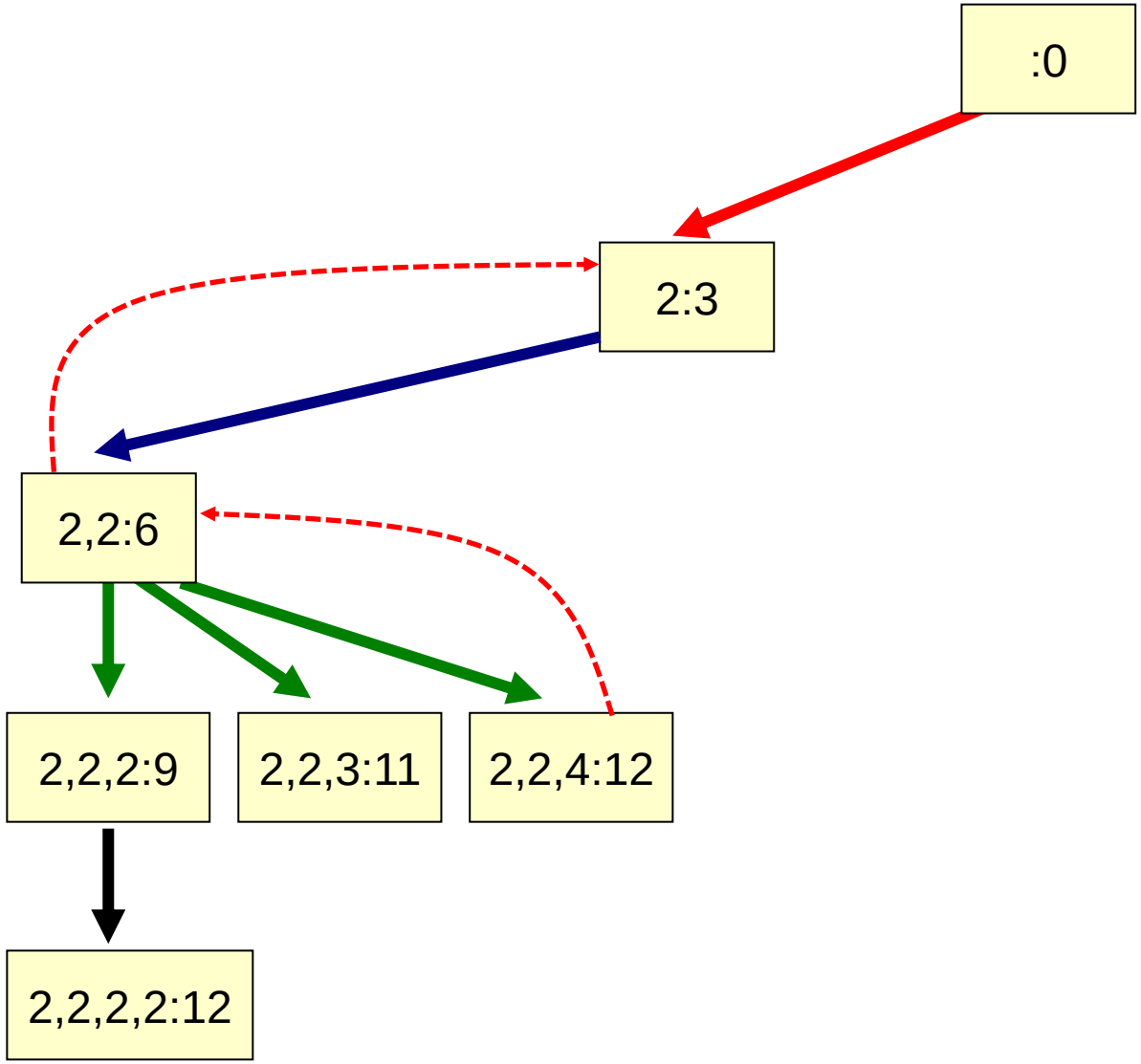


item	1	2	3	4
peso	2	3	4	5
valor	3	5	6	10

Peso máximo: 8

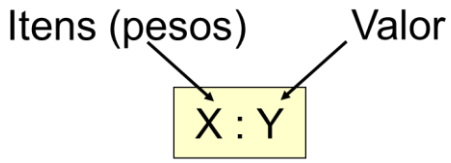


Backtracking

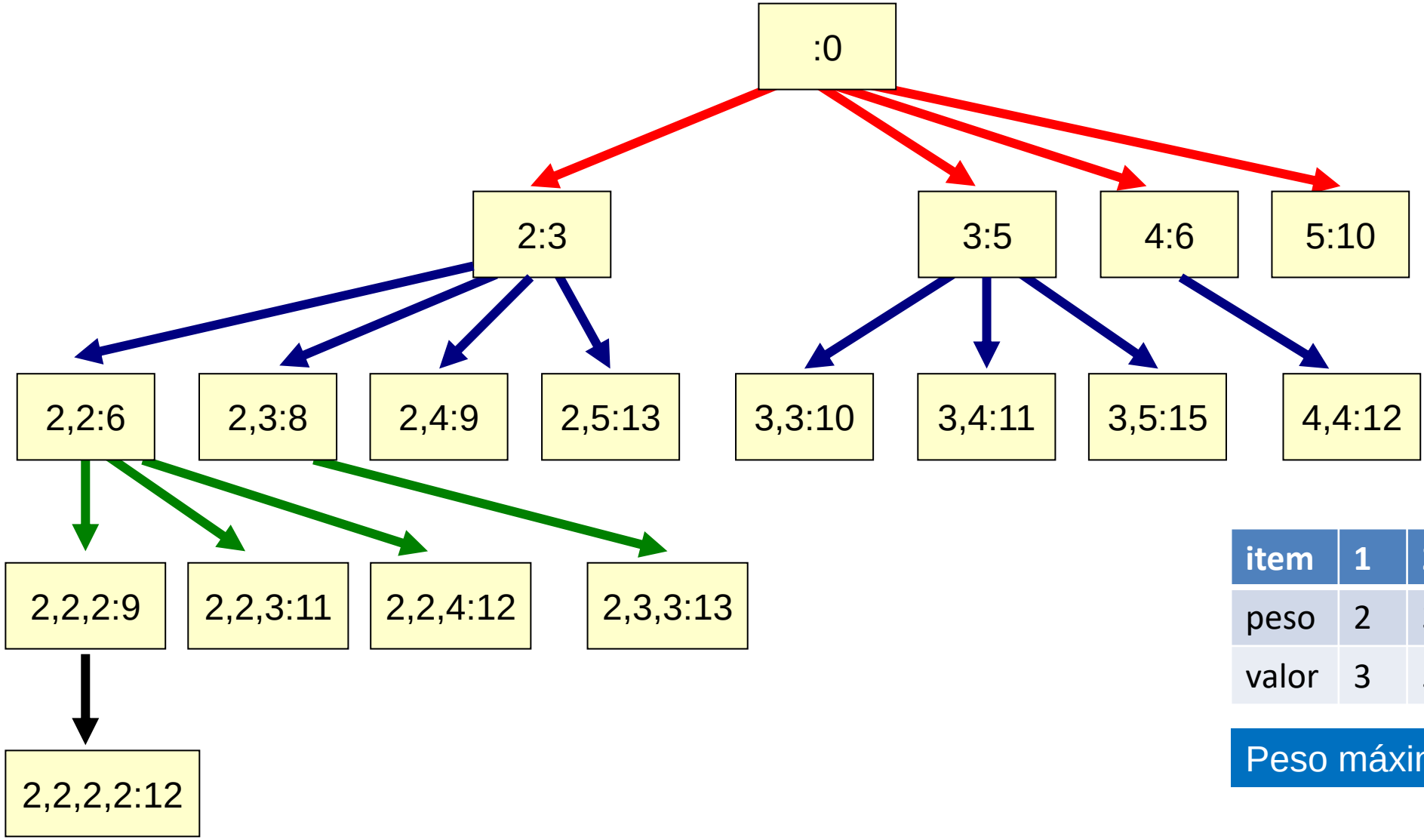


item	1	2	3	4
peso	2	3	4	5
valor	3	5	6	10

Peso máximo: 8



Backtracking



item	1	2	3	4
peso	2	3	4	5
valor	3	5	6	10

Peso máximo: 8

Backtracking - Mochila

W – peso máximo
n – número de itens
w[1..n] – vetor de pesos dos itens
v[1..n] – vetor de valores dos itens

Backpack (i, r) – Calcula o valor do melhor carregamento da mochila usando objetos do tipo i ao n, com carga máxima r.

Backtracking - Mochila

```
Backpack ( i,r )

b ← 0
 Para k  ←  i até n
    Se w [ k ]  ≤ r
       b ← max ( b, v [k] + Backpack ( k, r- w [k]) )
  retorna(b)
```

FIM